

Intégration des APIs CAMARA pour l'optimisation dynamique des réseaux via intelligence artificielle

Encadrant : **W. Diego,**

Etudiants : **S. Bitan,**

Table des matières

1	Cahier des charges	4
2	Plan de développement	5
2.1	Tâche 1 : Analyse de l'écosystème CAMARA et des APIs télécom	5
2.2	Tâche 2 : Conception et développement du middleware CAMARA et du serveur MCP	5
2.3	Tâche 3 : Développement et intégration des agents d'intelligence artificielle	6
2.4	Tâche 4 : Implémentation des cas d'usage et évaluation du système	6
3	Etat de l'art	7
3.1	Introduction	7
3.2	Softwarisation des réseaux et émergence du Network-as-a-Service	7
3.3	Exposition des capacités réseau via des API	8
3.3.1	Le rôle des interfaces programmables dans les réseaux modernes . .	8
3.3.2	Frameworks d'exposition des API réseau	8
3.4	Le projet CAMARA et les API télécom	9
3.4.1	Origine et objectifs du projet CAMARA	9
3.4.2	Architecture et fonctionnement des API CAMARA	9
3.5	Optimisation et orchestration intelligente des réseaux	10
3.5.1	Complexité croissante des réseaux modernes	10
3.5.2	Apport de l'intelligence artificielle dans la gestion des réseaux . . .	10
3.6	Architectures multi-agents fondées sur MCP	11
3.7	Synthèse et positionnement du projet	12
4	Analyse	13
4.1	Description générale et expression des besoins	13
4.2	Besoins fonctionnels et exigences non fonctionnelles	13
4.3	Choix d'architecture et contraintes techniques	14
4.4	Identification des problèmes et limites	15
5	Conception	16
5.1	Vue d'ensemble de l'architecture finale	16
5.2	Répartition des composants techniques	17
5.3	Conception du middleware CAMARA	18
5.4	Conception du serveur MCP	18
5.5	Conception de l'agent d'intelligence artificielle	20
5.6	Applications et cas d'usage	21
5.6.1	Optimisation dynamique de la connectivité avec Quality on Demand	21
5.6.2	Évaluation du risque avec SIM Swap et Device Status	21

6	Compte rendu du projet	22
6.1	Déroulement général du projet	22
6.2	Mise en place de l'environnement réseau et de la couche CAMARA	23
6.3	Développement du serveur MCP	24
6.4	Intégration de l'agent local	25
6.5	Intégration du second agent d'intelligence artificielle	25
6.6	Interface web de supervision et contrôle des mobiles	26
6.7	Validation des cas d'usage	27
6.7.1	Validation du cas Quality on Demand	27
6.7.2	Validation du cas SIM Swap et sécurité d'un paiement	27
6.8	Difficultés rencontrées et solutions apportées	27
6.9	Fonctionnement technique de bout en bout	28
6.10	Ajouts réalisés en fin de projet : serveurs MCP spécialisés et nouvelles routes CAMARA	29
6.10.1	Ajout de deux serveurs MCP spécialisés	30
6.10.2	Ajout des routes CAMARA OTP SMS	31
6.10.3	Ajout de la route CAMARA Subscriber Profile	32
6.10.4	Intérêt global des ajouts	33
6.11	Livraison finale et bilan	33
6.12	Conclusion	34
	Références	36
A	Annexes	38
A.1	Déploiement de free5GC	38
A.2	Terminaux enregistrés dans free5GC	38
A.3	Conteneur du serveur CAMARA	39
A.4	Appel direct à l'API SIM Swap CAMARA	39
A.5	Outils exposés par le serveur MCP	40
A.6	Appel d'un outil MCP par l'agent local	40
A.7	Création d'une session Quality on Demand	41
A.8	Consultation d'une session Quality on Demand	41
A.9	Interface web de supervision CAMARA	42

1 Cahier des charges

Le projet CAMARA, porté par la Linux Foundation et le GSMA Operator Platform Group, vise à standardiser l'exposition des capacités des réseaux télécoms à travers des APIs ouvertes, interopérables et multi-opérateurs. Ces interfaces permettent à des applications tierces d'accéder dynamiquement à différentes fonctions du réseau, telles que la localisation d'un terminal, l'état du dispositif, la qualité de service à la demande ou encore la détection de changement de carte SIM. Dans ce contexte, l'émergence d'agents d'intelligence artificielle capables d'orchestrer des services logiciels ouvre de nouvelles perspectives pour la gestion des réseaux. L'utilisation de protocoles d'intégration comme le Model Context Protocol permet d'envisager une interaction structurée entre un agent IA et des services réseau exposés via des APIs. Cette évolution s'inscrit dans la transformation des infrastructures télécoms vers des réseaux programmables et pilotés par logiciel.

L'objectif du projet est de concevoir et de déployer une architecture intégrant les APIs CAMARA avec un agent d'intelligence artificielle via un serveur MCP. Cette architecture devra permettre d'automatiser certaines décisions réseau en fonction du contexte observé. Le projet vise notamment à démontrer qu'un réseau télécom peut être utilisé comme une plateforme programmable capable d'adapter dynamiquement la qualité de service d'une application critique et de renforcer la sécurité d'une transaction sensible grâce à la corrélation d'informations issues du réseau.

Les travaux commenceront par une étude approfondie de l'architecture CAMARA et des principales APIs, notamment Device Status, Device Location, Quality on Demand et SIM Swap. Cette analyse portera sur leur modèle d'exposition, leurs mécanismes de sécurité ainsi que leur capacité d'interopérabilité entre différents opérateurs. Le rôle du Model Context Protocol sera également étudié afin de comprendre comment il permet d'organiser l'interaction entre l'agent IA et les services réseau. Une seconde phase consistera à concevoir et déployer une architecture technique basée sur des conteneurs intégrant les APIs CAMARA, un serveur MCP et un agent d'intelligence artificielle. Les services devront être configurés et orchestrés afin de permettre leur communication. L'agent IA devra être capable d'interpréter le contexte réseau et d'invoquer dynamiquement les APIs appropriées. Enfin, deux cas d'usage seront implémentés et testés dans un environnement contrôlé : l'activation dynamique d'une qualité de service prioritaire pour une application critique et la détection d'une tentative de fraude basée sur un changement de carte SIM. Des scénarios de simulation permettront d'évaluer la pertinence des décisions automatisées. Une analyse critique comparera l'approche API classique et l'approche pilotée par l'IA en termes de performance, de scalabilité, de sécurité et de pertinence industrielle.

2 Plan de développement

Le plan de développement initial a été globalement conservé, mais certaines adaptations ont été nécessaires au cours de la réalisation du projet. Ces ajustements concernent principalement l'environnement technique, l'intégration des composants et la manière de simuler certaines capacités réseau non directement disponibles dans Free5G. L'objectif général est resté inchangé : concevoir une architecture permettant à un agent d'intelligence artificielle d'exploiter des APIs CAMARA via un serveur MCP afin d'orchestrer des décisions liées à la qualité de service et à la sécurité.

Le projet s'organise autour de quatre tâches principales, structurées de manière progressive afin de passer d'une phase d'analyse théorique à une implémentation opérationnelle intégrant les différents composants du système.

2.1 Tâche 1 : Analyse de l'écosystème CAMARA et des APIs télécom

La première étape a consisté à étudier l'écosystème CAMARA et les APIs télécom associées au projet. L'analyse a porté principalement sur les APIs Quality on Demand, Device Status, Device Location et SIM Swap, ainsi que sur leurs mécanismes d'exposition, leurs paramètres et leurs contraintes d'utilisation.

Cette phase a également permis d'identifier les écarts entre les spécifications théoriques des APIs CAMARA et leur mise en œuvre dans un environnement expérimental. En particulier, certaines informations nécessaires aux scénarios de fraude, comme l'historique de changement de carte SIM, ne sont pas directement stockées par Free5G. Ce constat a conduit à prévoir des mécanismes de simulation dans le middleware CAMARA afin de pouvoir tester la chaîne fonctionnelle complète.

2.2 Tâche 2 : Conception et développement du middleware CAMARA et du serveur MCP

La deuxième tâche a porté sur la conception de la couche d'exposition des APIs et du serveur MCP. Le middleware CAMARA, développé en Node.js, a été conçu pour exposer des endpoints REST représentant les capacités réseau utilisées dans le projet. Il prend notamment en charge l'authentification, la création et la consultation de sessions Quality on Demand, la récupération du statut d'un terminal et la simulation des informations liées au SIM Swap.

En parallèle, un serveur MCP a été développé en Python afin de transformer ces capacités CAMARA en outils exploitables par un agent d'intelligence artificielle. Cette couche permet de masquer la complexité des appels REST, de normaliser les réponses au format JSON et de séparer clairement les outils d'observation des outils d'action. Le serveur MCP ne porte pas la logique de décision métier : il expose les outils, tandis que l'agent conserve la responsabilité du raisonnement et de la décision finale.

2.3 Tâche 3 : Développement et intégration des agents d'intelligence artificielle

La troisième tâche a consisté à intégrer les agents d'intelligence artificielle capables d'interagir avec le serveur MCP. Deux modes d'exécution ont été mis en place. Le premier repose sur un agent local utilisant Llama 3.2 via Ollama, afin de conserver les données dans l'environnement de test et de limiter la dépendance à des services externes. Le second repose sur un agent utilisant un modèle OpenAI via des tokens d'API, afin de comparer le comportement d'un modèle plus puissant avec celui du modèle local. Ces deux agents utilisent le même serveur MCP et les mêmes outils CAMARA. Cette organisation permet de vérifier que l'architecture reste modulaire et indépendante du modèle de langage utilisé. Le travail réalisé dans cette phase a également porté sur la définition des prompts, la description des outils disponibles, la structuration des réponses attendues et le contrôle des décisions produites par les agents.

2.4 Tâche 4 : Implémentation des cas d'usage et évaluation du système

La dernière tâche a consisté à intégrer l'ensemble des composants et à valider deux cas d'usage principaux. Le premier concerne l'activation dynamique d'une session Quality on Demand afin d'améliorer temporairement la qualité de service d'un terminal ou d'une application critique. Ce scénario permet de tester une API d'action, puisque l'appel CAMARA modifie l'état du système.

Le second cas d'usage porte sur l'évaluation du risque lors d'une transaction sensible. L'agent interroge plusieurs signaux réseau, notamment le statut du terminal et l'existence éventuelle d'un changement récent de carte SIM, puis produit une recommandation de risque. Ce scénario permet de tester l'utilisation des APIs CAMARA comme sources d'observation et d'aide à la décision.

Des scénarios de test ont été mis en place afin de vérifier la communication entre les différentes briques : agent IA, serveur MCP, middleware CAMARA et environnement Free5G. L'évaluation a porté sur la capacité de l'agent à sélectionner les bons outils, à interpréter correctement les réponses retournées et à produire une décision cohérente. Une analyse critique complète ensuit cette validation en comparant l'approche proposée à une orchestration classique fondée sur des règles statiques.

3 Etat de l'art

3.1 Introduction

Les réseaux de télécommunications connaissent une transformation profonde sous l'effet de la virtualisation, du cloud computing, de la 5G et de l'émergence de nouveaux besoins applicatifs. Les infrastructures réseau ne sont plus uniquement considérées comme des moyens de transport de données, mais comme des plateformes programmables capables d'exposer certaines de leurs capacités à des applications externes. Cette évolution s'inscrit dans une logique de Network-as-a-Service, dans laquelle des fonctions telles que la qualité de service, l'état d'un terminal, la localisation ou certains signaux de sécurité peuvent être consommés à travers des APIs.

Le projet CAMARA s'inscrit précisément dans cette dynamique. Porté par la Linux Foundation et associé à l'initiative GSMA Open Gateway, il vise à standardiser l'exposition des capacités télécoms sous forme d'APIs ouvertes, interopérables et plus simples à intégrer pour les développeurs [1, 2]. L'intérêt de cette approche est de réduire la complexité des réseaux télécoms tout en permettant à des applications externes d'interagir avec des capacités traditionnellement réservées aux opérateurs.

En parallèle, les progrès récents de l'intelligence artificielle, et en particulier des modèles de langage, ouvrent de nouvelles perspectives pour l'orchestration de ces APIs. Un agent IA peut interpréter une demande, sélectionner des outils, appeler des services externes et produire une décision contextualisée. L'objectif de cet état de l'art est donc d'étudier les fondements techniques permettant de relier les APIs CAMARA à des agents d'intelligence artificielle via le Model Context Protocol, dans le but d'automatiser des décisions liées à la qualité de service et à la sécurité.

3.2 Softwarisation des réseaux et émergence du Network-as-a-Service

Les réseaux de télécommunications ont longtemps reposé sur des équipements matériels spécialisés, dans lesquels les fonctions réseau étaient fortement couplées à l'infrastructure physique. Cette approche présentait des limites en matière de flexibilité, d'évolutivité et d'automatisation. Avec l'augmentation du trafic, la multiplication des services numériques et l'apparition de besoins très hétérogènes, les opérateurs ont progressivement adopté des architectures plus logicielles.

Deux évolutions ont joué un rôle central dans cette transformation : le Software Defined Networking et la Network Function Virtualization. Le SDN repose sur la séparation entre le plan de contrôle et le plan de données, ce qui permet de piloter le réseau de manière plus centralisée et programmable. La NFV consiste à exécuter des fonctions réseau sous forme logicielle, sur des infrastructures de calcul génériques, au lieu de dépendre uniquement d'équipements spécialisés. Ces approches ont contribué à rendre les réseaux plus flexibles et plus adaptés à l'automatisation [3].

La 5G prolonge cette évolution avec une architecture orientée services. Le cœur

de réseau 5G repose sur une Service-Based Architecture, dans laquelle les fonctions réseau communiquent au moyen d'interfaces standardisées. Cette organisation facilite l'intégration avec les environnements cloud et permet d'exposer certaines fonctionnalités sous forme de services consommables par d'autres composants [4]. Le network slicing illustre également cette logique, puisqu'il permet de créer plusieurs réseaux virtuels adaptés à des besoins spécifiques, par exemple pour les communications critiques, l'Internet des objets ou les services à très haut débit [5].

Dans ce contexte apparaît le concept de Network-as-a-Service. L'idée est de permettre à des applications externes de consommer certaines capacités réseau à la demande, à travers des interfaces programmables. Ordonez-Lucena et Dsouza montrent que CAMARA s'inscrit dans cette trajectoire en proposant des chemins concrets vers une exposition standardisée des capacités réseau [6]. Le réseau devient ainsi une plateforme de services, et non plus uniquement une infrastructure de connectivité.

3.3 Exposition des capacités réseau via des API

3.3.1 Le rôle des interfaces programmables dans les réseaux modernes

L'évolution des réseaux vers des architectures logicielles et virtualisées a conduit à l'émergence d'un nouveau paradigme dans lequel les capacités du réseau peuvent être exposées à des applications externes à travers des interfaces programmables. Ces interfaces, généralement désignées sous le terme d'API (*Application Programming Interfaces*), constituent un élément clé dans la transformation des infrastructures de télécommunications vers des plateformes de services ouvertes et interopérables.

Dans ce modèle, certaines fonctionnalités du réseau — telles que la gestion de la qualité de service, l'accès aux informations de localisation des terminaux ou encore le contrôle de certaines politiques réseau — peuvent être accessibles par des applications tierces. L'objectif est de permettre aux développeurs d'interagir avec l'infrastructure réseau sans avoir à manipuler directement les mécanismes internes complexes qui assurent son fonctionnement.

Le réseau n'est plus uniquement perçu comme un simple moyen de transport de données, mais comme un ensemble de services programmables pouvant être consommés à la demande par des applications externes. Cette évolution contribue à renforcer l'intégration entre les applications et l'infrastructure de communication, ouvrant la voie à de nouveaux cas d'usage dans des domaines tels que l'Internet des objets, les services multimédias à faible latence ou les applications industrielles critiques [2].

3.3.2 Frameworks d'exposition des API réseau

Afin de faciliter l'adoption des API réseau et d'en garantir l'interopérabilité, plusieurs initiatives de standardisation ont été proposées par différents organismes industriels et académiques. Parmi les plus importantes figure le *Common API Framework* (CAPIF), défini par le consortium 3GPP. Ce cadre architectural vise à fournir un environnement

sécurisé et standardisé pour la publication, la découverte et la consommation des API réseau [7].

Parallèlement, l'initiative *Multi-access Edge Computing* (MEC) développée par l'ETSI propose également un ensemble d'API permettant aux applications d'interagir avec les ressources disponibles à la périphérie du réseau. Ces interfaces offrent par exemple la possibilité d'accéder à des informations contextuelles relatives au réseau ou de déployer des services directement au niveau des infrastructures edge afin de réduire la latence des communications [8].

Ces différentes initiatives contribuent à définir un cadre technique commun permettant de simplifier l'exposition des capacités réseau. Mais, malgré ces efforts de standardisation, l'adoption de ces interfaces par les développeurs reste limitée par la complexité inhérente aux infrastructures télécom. Pour palier à ce problème, de nouvelles initiatives ont émergé afin de proposer des API plus accessibles et plus faciles à intégrer dans des environnements applicatifs modernes [9].

3.4 Le projet CAMARA et les API télécom

3.4.1 Origine et objectifs du projet CAMARA

Dans le prolongement des travaux visant à faciliter l'exposition des capacités réseau, le projet *CAMARA* constitue aujourd'hui l'une des initiatives les plus importantes dans le domaine des API télécom. Lancé conjointement par la Linux Foundation et la GSMA, ce projet open source vise à définir un ensemble d'API standardisées permettant aux développeurs d'accéder aux fonctionnalités des réseaux de télécommunications de manière simple et uniforme [6].

L'objectif principal de CAMARA est de combler l'écart entre les infrastructures télécom et les environnements de développement applicatif modernes. En effet, les réseaux mobiles reposent sur des architectures complexes et sur des standards techniques souvent difficiles à appréhender pour des développeurs ne disposant pas d'une expertise approfondie dans le domaine des télécommunications.

3.4.2 Architecture et fonctionnement des API CAMARA

L'architecture du projet CAMARA repose sur plusieurs composants principaux permettant de faire le lien entre les applications externes et les fonctions internes du réseau. Les API exposées aux applications sont appelées *Service APIs*, tandis que les interfaces directement associées aux composants réseau sont désignées sous le terme de *Network APIs*. Une couche intermédiaire assure la transformation des requêtes applicatives en opérations réseau correspondantes [10].

Dans cette architecture, les requêtes émises par les applications sont généralement traitées par une passerelle d'API (*API Gateway*) chargée de gérer les mécanismes d'authentification, d'autorisation et de journalisation des interactions. Cette passerelle constitue un point d'entrée sécurisé permettant de contrôler l'accès aux différentes capacités du

réseau. Les requêtes sont ensuite traduites en appels vers les fonctions réseau appropriées avant que les réponses ne soient renvoyées aux applications clientes.

Plusieurs API ont déjà été définies dans le cadre du projet CAMARA afin de répondre à différents cas d'usage. Parmi celles-ci figurent notamment les API de gestion de la qualité de service (*Quality on Demand*), les API de localisation des terminaux, ou encore les interfaces permettant d'accéder à certaines capacités des infrastructures edge [1]. Ces interfaces permettent aux applications d'adapter dynamiquement leur comportement en fonction de l'état du réseau ou des besoins spécifiques des utilisateurs.

En facilitant l'accès aux capacités avancées des réseaux de nouvelle génération, le projet CAMARA contribue ainsi à transformer les infrastructures de télécommunications en plateformes programmables capables de soutenir un large éventail de services numériques.

3.5 Optimisation et orchestration intelligente des réseaux

3.5.1 Complexité croissante des réseaux modernes

La complexité des réseaux modernes rend difficile une gestion entièrement manuelle ou fondée uniquement sur des règles statiques. Les infrastructures 5G doivent gérer des services hétérogènes, des besoins variables en qualité de service, des contraintes de latence et des exigences de sécurité.

L'une des principales innovations introduites par la 5G est le concept de *network slicing*, qui permet de créer plusieurs réseaux virtuels logiquement isolés au sein d'une même infrastructure physique. Chaque tranche réseau peut être configurée afin de répondre aux besoins spécifiques d'un type d'application donné, par exemple les communications ultra-fiables à faible latence, les services multimédias à très haut débit ou les communications massives entre objets connectés [5]. Si cette approche offre une grande flexibilité, elle nécessite également des mécanismes de gestion et d'orchestration capables de s'adapter en permanence à des conditions de trafic dynamiques et à des exigences de qualité de service variées.

3.5.2 Apport de l'intelligence artificielle dans la gestion des réseaux

Face à ces défis, l'utilisation de techniques d'intelligence artificielle et d'apprentissage automatique suscite un intérêt croissant dans le domaine des réseaux de communication. Ces approches permettent d'automatiser certaines tâches de gestion réseau et d'améliorer l'efficacité des mécanismes d'orchestration en s'appuyant sur l'analyse de données issues de l'infrastructure.

Les méthodes d'apprentissage automatique peuvent notamment être utilisées pour prédire l'évolution du trafic, optimiser l'allocation des ressources réseau ou détecter des anomalies dans le fonctionnement des infrastructures. Dans le contexte du *network slicing*, plusieurs travaux ont montré que les techniques d'apprentissage par renforcement pouvaient être employées afin d'adapter dynamiquement la configuration des tranches réseau en fonction des besoins des applications et de l'état du réseau [11].

Plus récemment, l'émergence des modèles de langage de grande taille (*Large Language Models*, LLM) et des architectures d'agents intelligents ouvre de nouvelles perspectives pour l'automatisation des réseaux. Dans ce cadre, certains travaux proposent d'utiliser des agents intelligents capables d'interpréter les intentions exprimées par les applications, d'analyser les métriques réseau et d'ajuster dynamiquement les paramètres de l'infrastructure afin d'optimiser les performances globales du système [11].

Ces approches s'inscrivent dans la vision plus large des réseaux autonomes, parfois désignés sous le terme de *self-driving networks*, dans lesquels les mécanismes d'intelligence artificielle jouent un rôle central dans la gestion et l'orchestration des ressources.

Dans un environnement où les capacités du réseau sont exposées à travers des interfaces programmables, les API réseau constituent un point d'interaction privilégié entre les applications et les mécanismes de gestion de l'infrastructure. Les systèmes d'intelligence artificielle peuvent ainsi exploiter ces interfaces afin d'agir directement sur le comportement du réseau.

Par exemple, un agent intelligent peut analyser les conditions de trafic, identifier une dégradation potentielle de la qualité de service et déclencher dynamiquement un appel à une API réseau afin d'ajuster certains paramètres de configuration. Dans le cadre des API proposées par le projet *CAMARA*, cela peut notamment se traduire par la modification des paramètres de qualité de service, l'accès à des informations de localisation des terminaux ou l'activation de certaines fonctionnalités réseau spécifiques [6].

3.6 Architectures multi-agents fondées sur MCP

Les architectures récentes d'agents autonomes ne se limitent plus à l'utilisation d'un unique modèle de langage connecté à quelques outils externes. Une tendance importante consiste au contraire à organiser plusieurs agents spécialisés autour d'une couche d'orchestration commune. Cette approche permet de décomposer une tâche complexe en sous-tâches plus simples, confiées à des agents dotés de rôles distincts. Dans ce type d'architecture, le Model Context Protocol (MCP) occupe une place particulière : il ne constitue pas le mécanisme de décision principal, mais fournit une interface standardisée entre les agents et les ressources externes, telles que des API, des bases de données, des outils ou des journaux d'exécution.

Dans un premier article il était proposé une architecture multi-agents hiérarchique dans laquelle plusieurs agents spécialisés partagent leurs états internes au moyen d'un serveur MCP centralisé [12]. Dans leur approche, plusieurs agents produisent des journaux structurés comprenant leur raisonnement, leur niveau de confiance et leur auto-évaluation. Ces informations peuvent être partagées via un serveur MCP, ce qui permet aux agents de tenir compte des hypothèses et incertitudes formulées par les autres. Même si leur expérimentation porte sur un domaine différent, elle montre l'intérêt de MCP comme support d'un contexte partagé, structuré et réutilisable dans une architecture multi-agents.

Cette logique est pertinente pour notre projet. Dans le cas des APIs CAMARA, plusieurs agents peuvent consommer les mêmes outils MCP tout en ayant des rôles ou des capacités différentes. Par exemple, un agent local peut être utilisé pour garantir la confi-

dentialité des données, tandis qu'un agent utilisant un modèle plus puissant peut être mobilisé pour comparer la qualité du raisonnement ou la pertinence des décisions. Le serveur MCP joue alors le rôle d'interface commune : il expose les outils CAMARA, mais ne dépend pas d'un modèle unique.

Une approche complémentaire combinant le protocole Agent-to-Agent (A2A) et MCP dans un scénario de réponse à incident informatique a été envisagé dans un second article [13]. Leur architecture distingue trois couches : une couche agents, une couche d'intégration protocolaire et une couche d'interaction avec les outils. Les agents spécialisés y jouent des rôles différents, notamment un agent coordinateur d'incident, un agent de diagnostic et un agent de résolution. A2A est utilisé pour la communication entre agents, la découverte de capacités, la délégation de tâches et le suivi de l'état des actions. MCP, de son côté, fournit la couche de contexte et l'accès standardisé aux outils opérationnels, tels que l'analyse de journaux, la supervision système, la base de connaissances ou le système de déploiement.

L'intérêt de MCP est donc double. D'une part, il découple les agents des APIs techniques sous-jacentes : l'agent n'a pas besoin de connaître directement les endpoints REST, les tokens ou les formats internes des APIs CAMARA. D'autre part, il rend possible l'utilisation de plusieurs agents sur un même ensemble d'outils. Cette propriété a été exploitée dans notre architecture, avec un agent local basé sur Llama 3.2 via Ollama et un second agent utilisant un modèle OpenAI via API. Les deux agents peuvent interroger le même serveur MCP, ce qui démontre la modularité de l'approche.

3.7 Synthèse et positionnement du projet

L'état de l'art montre que les réseaux télécoms évoluent vers des architectures programmables, orientées services et capables d'exposer certaines de leurs capacités à des applications externes. Les concepts de SDN, NFV, Service-Based Architecture, edge computing et Network-as-a-Service ont progressivement rendu possible cette ouverture. CAMARA et GSMA Open Gateway prolongent cette dynamique en proposant des APIs standardisées, interopérables et accessibles aux développeurs.

Les APIs CAMARA utilisées dans ce projet couvrent deux types de besoins. Quality on Demand illustre une API d'action permettant d'adapter temporairement la qualité de service. Device Status, Device Location et SIM Swap relèvent davantage de l'observation et fournissent des signaux utiles à la supervision ou à la détection de fraude. La valeur de l'approche ne vient donc pas uniquement de l'appel à une API isolée, mais de la capacité à corréler plusieurs informations pour produire une décision.

L'intégration d'agents IA via MCP constitue une réponse à ce besoin d'orchestration. MCP permet d'exposer les APIs CAMARA comme des outils structurés, utilisables par un ou plusieurs agents. Cette architecture conserve une séparation claire entre les capacités réseau, la couche d'exposition des outils et la logique de décision. Le projet se positionne ainsi à l'intersection de trois évolutions : la programmabilité des réseaux, la standardisation des APIs télécoms et l'émergence d'agents IA capables d'orchestrer des services externes.

4 Analyse

4.1 Description générale et expression des besoins

Le projet s'inscrit dans le contexte de l'évolution des réseaux télécoms vers des architectures programmables, dans lesquelles certaines capacités réseau peuvent être exposées à des applications externes à travers des APIs standardisées. Les APIs CAMARA permettent notamment d'accéder à des fonctions telles que la qualité de service à la demande, l'état d'un terminal, la localisation ou encore la vérification d'un changement récent de carte SIM. Toutefois, l'utilisation directe de ces APIs reste complexe pour un agent applicatif, car elle implique de gérer les endpoints REST, l'authentification, les formats de réponse, les erreurs et les contraintes propres à l'environnement réseau.

Le besoin principal du projet consiste donc à créer une couche d'intermédiation entre les capacités télécoms exposées par CAMARA et un agent d'intelligence artificielle capable de raisonner sur ces capacités. L'objectif n'est pas simplement d'appeler une API depuis un programme, mais de permettre à un agent de comprendre une demande formulée en langage naturel, de choisir les outils appropriés, d'interroger les APIs nécessaires et de produire une décision contextualisée.

Cette problématique fait apparaître trois besoins complémentaires. Le premier est un besoin d'abstraction : l'agent ne doit pas manipuler directement la complexité technique des APIs CAMARA. Le deuxième est un besoin d'orchestration : l'agent doit pouvoir combiner plusieurs signaux réseau afin de produire une réponse pertinente. Le troisième est un besoin de contrôle : les actions réseau sensibles, comme la création d'une session Quality on Demand, doivent être distinguées des opérations d'observation, comme la récupération du statut d'un terminal.

Deux cas d'usage ont été retenus afin de matérialiser ces besoins. Le premier, Smart Connectivity Boost, concerne l'activation dynamique d'une qualité de service renforcée pour un terminal ou une application critique. Il s'agit d'un cas d'usage orienté action, car l'appel à l'API Quality on Demand modifie l'état du système. Le second, Telco-Grade Fraud Prevention, concerne l'évaluation du risque lors d'une transaction sensible. Il repose sur l'analyse de signaux tels que le statut du terminal et l'existence éventuelle d'un SIM Swap récent. Ce second cas d'usage est davantage orienté observation et aide à la décision.

4.2 Besoins fonctionnels et exigences non fonctionnelles

Sur le plan fonctionnel, le système doit d'abord permettre l'accès aux principales capacités CAMARA retenues dans le projet. Il doit être possible de créer et consulter une session Quality on Demand, de récupérer l'état réseau d'un terminal, de vérifier un éventuel changement de carte SIM et, lorsque le scénario le nécessite, d'intégrer des informations de localisation. Ces capacités doivent être accessibles à travers des outils MCP clairement définis, afin que l'agent puisse les invoquer sans connaître directement les détails des APIs REST sous-jacentes.

Le système doit également permettre l'agrégation de plusieurs signaux réseau dans

un contexte unique. Cette exigence est particulièrement importante pour le cas d’usage de détection de fraude. En effet, un signal isolé ne suffit pas toujours à produire une décision fiable. Un changement récent de carte SIM peut être un indicateur de risque, mais son interprétation dépend aussi d’autres informations, comme l’état de connectivité du terminal ou la cohérence de sa localisation. L’agent doit donc pouvoir raisonner à partir d’un ensemble structuré de données.

Un autre besoin fonctionnel concerne la capacité à utiliser plusieurs agents sur un même serveur MCP. L’architecture doit rester indépendante du modèle de langage utilisé. Dans le cadre du projet, deux agents ont été mis en place : un agent local utilisant Llama 3.2 via Ollama, et un agent utilisant un modèle OpenAI à travers des tokens d’API. Cette exigence permet de comparer deux modes d’exécution tout en conservant le même serveur MCP et les mêmes outils CAMARA.

Sur le plan non fonctionnel, plusieurs exigences doivent être prises en compte. La première concerne la confidentialité des données. Les informations manipulées peuvent inclure des numéros de téléphone, des statuts réseau ou des signaux liés à une transaction. L’utilisation d’un agent local répond partiellement à cette contrainte, car elle permet de conserver les données dans l’environnement de test. La seconde exigence concerne la traçabilité. Les appels aux outils, les réponses retournées par les APIs et les décisions produites par l’agent doivent pouvoir être observés afin de faciliter la validation et le débogage.

La robustesse constitue également une contrainte importante. Le système doit gérer les erreurs d’appel API, les réponses incomplètes, les indisponibilités temporaires et les cas où certaines informations ne sont pas disponibles dans l’environnement Free5G. Enfin, le comportement de l’agent doit être encadré par des descriptions d’outils précises et par des réponses structurées, afin de limiter les interprétations incorrectes ou les décisions incohérentes.

4.3 Choix d’architecture et contraintes techniques

L’architecture retenue repose sur une séparation claire entre la couche réseau, la couche d’exposition des APIs et la couche décisionnelle. Cette séparation répond à un besoin de modularité et permet de rapprocher le projet d’un scénario réaliste, dans lequel une application externe ne dialogue pas directement avec le cœur réseau, mais avec une couche d’exposition contrôlée.

Dans une première approche, l’utilisation d’un environnement Open5G avait été envisagée. Cependant, les contraintes d’installation, de stabilité et de temps ont conduit à privilégier Free5G pour la réalisation finale. Free5G permet de disposer d’un environnement réseau expérimental plus accessible pour tester les interactions avec les composants du projet. Une machine virtuelle dédiée héberge ainsi l’environnement Free5G et le middleware CAMARA développé en Node.js. Ce middleware expose les endpoints nécessaires aux cas d’usage et permet de faire le lien entre les appels applicatifs et l’environnement réseau simulé.

La couche décisionnelle est séparée sur une autre machine virtuelle. Elle regroupe le

serveur MCP, le client agent et les modèles d'intelligence artificielle utilisés. Le serveur MCP est développé en Python et joue le rôle d'interface entre les agents et les APIs CAMARA. Il transforme les capacités réseau en outils exploitables par l'agent, normalise les réponses et masque la complexité technique des appels REST. Ce choix d'architecture permet de ne pas placer la logique de décision dans le serveur MCP. Le serveur expose des outils et fournit des données structurées, mais il ne décide pas à la place de l'agent. Cette séparation est essentielle pour conserver l'intérêt de l'approche IA : l'agent doit choisir les outils à invoquer, interpréter les résultats et produire la conclusion finale. En revanche, le serveur MCP conserve un rôle de contrôle technique, notamment en distinguant les outils d'observation des outils d'action. Certaines contraintes techniques ont également influencé la conception. Free5G ne stocke pas directement toutes les informations métier nécessaires aux scénarios CAMARA, notamment l'historique de changement de carte SIM. Pour permettre la validation du cas d'usage SIM Swap, une simulation de ces informations a donc été intégrée au middleware. Cette adaptation ne remet pas en cause l'objectif du projet, car elle permet de tester la chaîne complète d'orchestration entre l'utilisateur, l'agent IA, le serveur MCP, le middleware CAMARA et l'environnement réseau.

4.4 Identification des problèmes et limites

La première limite identifiée concerne l'utilisation d'un modèle de langage dans un contexte décisionnel. Contrairement à un programme déterministe, un agent IA peut produire des réponses variables selon la formulation de la demande, le prompt utilisé ou le modèle sélectionné. Cette variabilité impose de contrôler la manière dont les outils sont décrits et la forme des réponses attendues. Il est donc nécessaire d'encadrer l'autonomie de l'agent sans supprimer sa capacité de raisonnement.

La deuxième limite concerne la distinction entre observation et action. Certaines APIs, comme Device Status ou SIM Swap, fournissent des informations utiles à l'analyse. D'autres, comme Quality on Demand, peuvent modifier l'état du système. Cette différence impose une vigilance particulière : une action réseau ne doit pas être déclenchée de manière implicite ou ambiguë. L'architecture doit donc rendre explicite le moment où l'agent passe d'une phase d'analyse à une phase d'action.

La troisième limite concerne l'environnement de test. Le projet repose sur une infrastructure expérimentale et non sur un réseau opérateur réel. Certaines données doivent être simulées, et les performances observées ne peuvent pas être généralisées directement à un environnement industriel. Les résultats permettent donc de valider la faisabilité de l'approche, mais pas de conclure définitivement sur sa robustesse à grande échelle.

Enfin, la comparaison entre l'agent local et l'agent utilisant un modèle OpenAI montre un compromis important. Le modèle local présente un avantage en matière de confidentialité et de maîtrise de l'environnement, mais peut être plus limité dans la qualité du raisonnement. À l'inverse, un modèle distant plus puissant peut produire de meilleures analyses, mais introduit une dépendance externe et soulève davantage de questions liées à la protection des données. Cette limite confirme l'intérêt d'une architecture découplée, dans laquelle le serveur MCP reste identique quel que soit le modèle utilisé.

5 Conception

5.1 Vue d'ensemble de l'architecture finale

La conception finale du projet repose sur une architecture distribuée séparant clairement la couche télécom, la couche d'exposition des APIs, la couche décisionnelle et la couche de supervision. Cette séparation permet de reproduire un scénario réaliste dans lequel une application externe ou un agent intelligent consomme des capacités réseau exposées par un opérateur à travers des APIs standardisées.

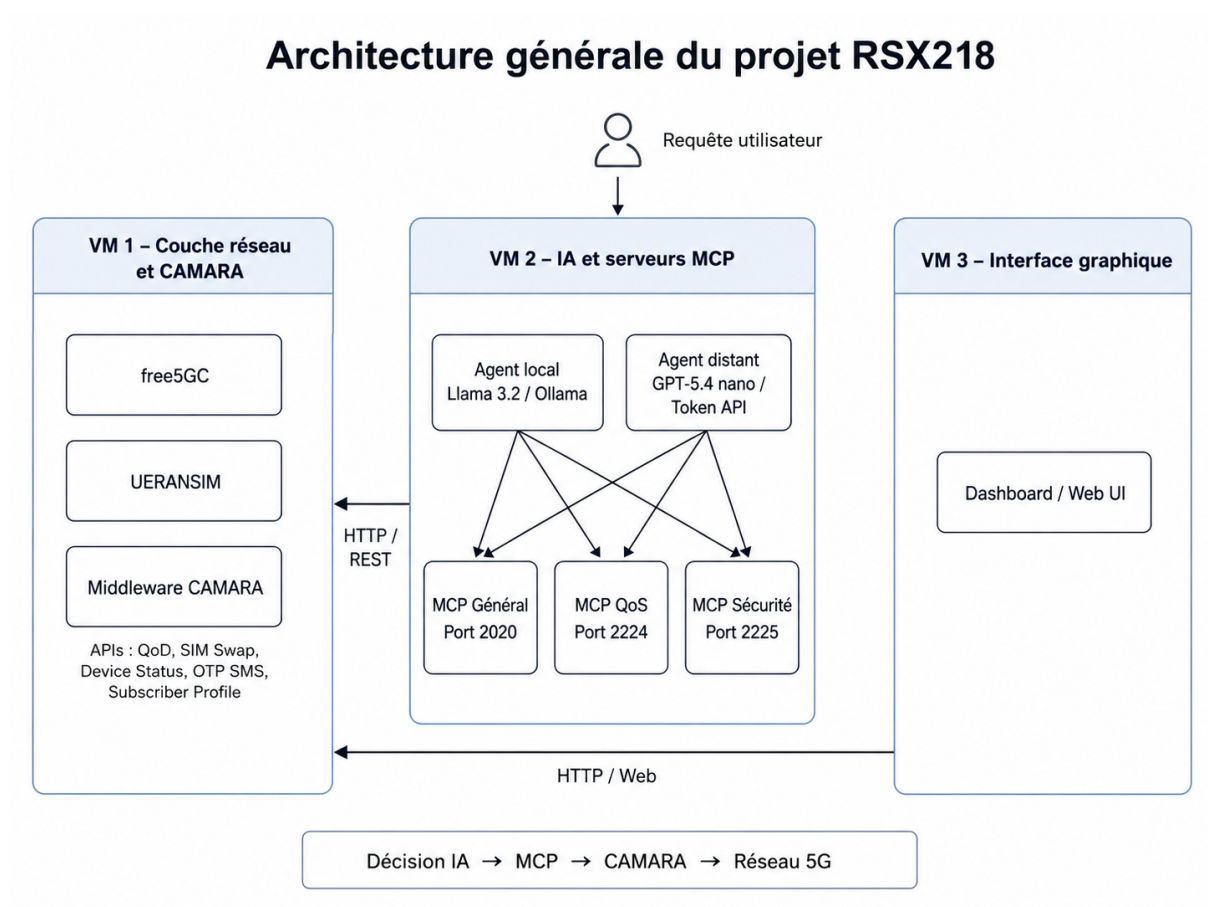


FIGURE 1 : Architecture générale de la solution intégrant le réseau 5G, les APIs CAMARA, les serveurs MCP, les agents d'intelligence artificielle et l'interface graphique

Comme l'illustre la Figure 1, l'architecture finale est répartie sur trois machines virtuelles. La première machine virtuelle héberge la couche réseau et la couche d'exposition des capacités télécoms. Elle regroupe le cœur de réseau free5GC, UERANSIM, utilisé pour simuler les terminaux mobiles, ainsi que le middleware CAMARA chargé d'exposer les différentes capacités réseau sous forme d'APIs REST.

La deuxième machine virtuelle héberge la couche décisionnelle. Elle comprend deux agents d'intelligence artificielle : un agent local reposant sur Llama 3.2 exécuté avec Ollama et un agent distant utilisant un modèle OpenAI à travers un token d'API. Ces agents peuvent sélectionner l'un des trois serveurs MCP selon la nature de la demande. Le serveur MCP général, accessible sur le port 2020, regroupe l'ensemble des outils. Le serveur MCP QoS, exposé sur le port 2224, est spécialisé dans les opérations liées à Quality on Demand. Enfin, le serveur MCP Sécurité, accessible sur le port 2225, regroupe les outils associés à la détection de fraude, au SIM Swap, au statut du terminal, à la localisation et à l'authentification OTP.

La troisième machine virtuelle fournit l'interface graphique de supervision. Elle permet de visualiser les informations relatives aux terminaux, aux sessions QoS, aux changements de carte SIM et aux différents signaux de sécurité. Cette interface communique avec la couche CAMARA au moyen de requêtes HTTP afin d'afficher l'état courant du système.

Le cheminement principal d'une requête commence lorsqu'un utilisateur formule une demande en langage naturel. L'agent analyse cette demande, sélectionne le serveur MCP et les outils appropriés, puis le serveur MCP traduit cette décision en appels HTTP vers le middleware CAMARA. Celui-ci interagit ensuite avec l'environnement réseau free5GC ou avec les données simulées associées aux APIs. Le résultat suit ensuite le chemin inverse jusqu'à l'agent, qui l'interprète et présente une réponse contextualisée à l'utilisateur. L'architecture suit ainsi la chaîne : *décision IA, outil MCP, API CAMARA, capacité réseau 5G*.

5.2 Répartition des composants techniques

L'architecture du projet repose sur une répartition des composantes réseau, applicatives et décisionnelles. L'environnement Free5G constitue la base du réseau mobile simulé. Il permet de représenter un cœur réseau 5G expérimental, de créer ou superviser des terminaux, et d'observer certains états associés aux équipements connectés.

Au-dessus de cet environnement, un middleware CAMARA a été développé en Node.js. Ce composant expose des endpoints REST inspirés des APIs CAMARA utilisées dans le projet. Il prend en charge les appels liés à l'authentification, à la qualité de service à la demande, au statut du terminal et à la détection de changement de carte SIM. Le middleware joue donc le rôle d'interface entre l'environnement réseau simulé et les composants applicatifs externes.

Le serveur MCP est développé en Python. Il expose les fonctionnalités CAMARA sous forme d'outils appelables par un agent. Chaque outil correspond à une capacité précise : vérifier un changement de SIM, récupérer l'état d'un terminal, agréger plusieurs signaux de sécurité ou créer une session Quality on Demand. Cette couche permet à l'agent de ne pas manipuler directement les endpoints REST, les tokens d'authentification ou les formats propres aux APIs CAMARA.

Enfin, l'agent d'intelligence artificielle est exécuté sur la machine dédiée à la logique applicative. Il interagit avec le serveur MCP, sélectionne les outils nécessaires et produit une réponse finale à partir des résultats obtenus.

5.3 Conception du middleware CAMARA

Le middleware CAMARA constitue la couche d'exposition des capacités réseau. Il a été conçu comme une API REST intermédiaire permettant de représenter les principales fonctionnalités CAMARA utilisées dans le projet.

Le middleware intègre d'abord un mécanisme d'authentification inspiré du fonctionnement OAuth2. Un endpoint permet de générer un token d'accès utilisé ensuite par les autres requêtes. Cela permet de conserver une logique proche des APIs opérateur réelles, dans lesquelles les applications clientes doivent s'authentifier avant de consommer des capacités réseau.

Pour le cas d'usage Quality on Demand, le middleware expose des routes permettant de créer une session QoD, de récupérer les informations associées à une session existante et de supprimer une session. Ces routes permettent de simuler l'activation temporaire d'un profil de qualité de service pour un terminal ou une application critique.

Pour le cas d'usage de détection de fraude, le middleware expose des capacités d'observation, notamment la vérification d'un éventuel SIM Swap et la récupération du statut réseau d'un terminal. Dans la mesure où Free5G ne stocke pas directement certaines informations métier, comme l'historique de changement de carte SIM, une partie de ces données est simulée dans le middleware. Cette simulation permet néanmoins de tester le raisonnement de l'agent IA et la chaîne d'appel complète entre l'utilisateur, le serveur MCP, CAMARA et l'environnement réseau.

5.4 Conception du serveur MCP

Le serveur MCP constitue la couche d'intégration entre les agents d'intelligence artificielle et les capacités réseau exposées par CAMARA. Son rôle n'est pas de porter la logique métier de décision, mais de transformer les APIs CAMARA en outils techniques structurés, documentés et directement exploitables par un agent. Il agit ainsi comme une couche d'abstraction entre le raisonnement de l'agent et les appels REST réalisés vers le middleware CAMARA.

Chaque outil MCP encapsule un ou plusieurs appels HTTP vers les endpoints CAMARA. Le serveur reçoit une demande de l'agent, valide les paramètres transmis, construit la requête vers le middleware CAMARA, récupère la réponse JSON, puis la reformate dans une structure plus simple à interpréter par le modèle de langage. Cette organisation permet à l'agent de ne pas manipuler directement les URLs, les tokens d'authentification, les formats internes des APIs ou les erreurs techniques propres aux appels réseau.

Une distinction importante a été introduite entre les outils d'observation, les outils d'action et les outils d'agrégation. Les outils d'observation permettent uniquement de récupérer des informations, par exemple l'état d'un terminal, la présence d'un changement récent de carte SIM ou la cohérence d'une localisation. Les outils d'action déclenchent une modification de l'état du système, comme la création, la prolongation ou la suppression d'une session Quality on Demand. Enfin, les outils d'agrégation combinent plusieurs si-

gnaux CAMARA afin de fournir à l'agent un contexte déjà structuré pour sa prise de décision.

Cette séparation est essentielle dans le cadre du projet. Une opération comme `check_sim_swap` ou `get_device_status` ne modifie pas l'état du réseau : elle fournit uniquement un signal exploitable par l'agent. À l'inverse, une opération comme `create_qod_session` déclenche une action réseau simulée dans l'environnement CAMARA. Le serveur MCP permet donc d'encadrer techniquement l'autonomie de l'agent en rendant explicite la nature de chaque outil.

Le serveur MCP expose plusieurs outils pouvant être classés selon leur fonction. Les outils d'observation permettent de consulter les informations retournées par les APIs CAMARA sans modifier l'état du système. Les outils d'action déclenchent une opération, tandis que l'outil d'agrégation combine plusieurs signaux afin de fournir un contexte exploitable par l'agent.

Outil MCP et API	Description
<code>check_sim_swap</code> <i>SIM Swap</i>	Vérifie si un changement récent de carte SIM est associé au numéro de téléphone fourni.
<code>get_device_status</code> <i>Device Status</i>	Récupère l'état de connexion et la joignabilité du terminal.
<code>verify_location</code> <i>Device Location</i>	Vérifie si le terminal se trouve dans une zone géographique attendue.
<code>retrieve_location</code> <i>Device Location</i>	Récupère ou simule la position d'un terminal dans l'environnement de test.
<code>get_otp_status</code> <i>OTP SMS</i>	Consulte l'état d'une authentification OTP : en attente, validée, expirée ou échouée.
<code>get_subscriber_profile</code> <i>Subscriber Profile</i>	Récupère le profil d'un abonné à partir de son numéro de téléphone.
<code>get_qod_profile</code> <i>Quality on Demand / 5QI</i>	Récupère les profils de qualité de service disponibles et leur correspondance avec les profils 5QI.
<code>get_qod_session</code> <i>Quality on Demand</i>	Consulte l'état d'une session QoD à partir de son identifiant.

TABLE 1 : Outils MCP d'observation exposés aux agents

Outil MCP et API	description
Outil d'agrégation	
<code>get_payment_secur</code> <i>SIM Swap, Device Status et Location</i>	Regroupe plusieurs signaux réseau afin de fournir à l'agent un contexte structuré pour l'évaluation du risque.
Outils d'action	
<code>send_otp_code</code> <i>OTP SMS</i>	Crée une demande d'authentification et simule l'envoi d'un code OTP par SMS.
<code>validate_otp_code</code> <i>OTP SMS</i>	Vérifie le code transmis par l'utilisateur à partir de l'identifiant d'authentification.
<code>create_qod_session</code> <i>Quality on Demand</i>	Crée une session de qualité de service pour un terminal ou une application critique.
<code>extend_qod_session</code> <i>Quality on Demand</i>	Prolonge la durée d'une session Quality on Demand existante.
<code>delete_qod_session</code> <i>Quality on Demand</i>	Supprime une session Quality on Demand active ou arrivée en fin d'utilisation.

TABLE 2 : Outils MCP d'agrégation et d'action exposés aux agents

L'intérêt de cette couche MCP est donc triple. Premièrement, elle découple les agents des détails techniques des APIs CAMARA. Deuxièmement, elle normalise les réponses afin de faciliter leur interprétation par le modèle de langage. Troisièmement, elle rend l'architecture modulaire : le même ensemble d'outils peut être utilisé par plusieurs agents, qu'il s'agisse d'un modèle local exécuté avec Ollama ou d'un modèle externe plus performant.

5.5 Conception de l'agent d'intelligence artificielle

L'agent d'intelligence artificielle constitue la couche de raisonnement du système. Il reçoit une demande formulée en langage naturel, identifie l'objectif de l'utilisateur, sélectionne les outils MCP nécessaires, interprète les résultats retournés par les APIs CAMARA et produit une réponse finale justifiée. Contrairement à une approche fondée uniquement sur des règles statiques, l'agent peut adapter son raisonnement selon le contexte de la demande et les signaux réseau disponibles.

Deux modes d'intégration ont été mis en place afin d'évaluer la flexibilité de l'architecture. Le premier repose sur un agent local utilisant Llama 3.2 exécuté via Ollama. Ce choix permet de conserver les données dans l'environnement de test, sans dépendre d'un service externe. Il est particulièrement pertinent dans un contexte télécom, où les informations manipulées peuvent être sensibles, comme les numéros de téléphone, les statuts réseau ou les signaux liés à une transaction.

Le second mode repose sur un agent utilisant un modèle OpenAI accessible à l'aide de tokens d'API. Ce choix permet de comparer le comportement d'un modèle local avec celui d'un modèle plus puissant, notamment en matière de compréhension de la demande, de sélection des outils et de qualité de justification. Les deux agents utilisent le même serveur

MCP et les mêmes outils CAMARA, ce qui montre que l'architecture est découplée du modèle d'intelligence artificielle utilisé.

Une attention particulière a été portée à l'équilibre entre autonomie et contrôle. L'objectif n'est pas d'imposer une décision entièrement prédéfinie à l'agent, car cela supprimerait l'intérêt de l'approche IA. En revanche, l'agent doit être encadré par des descriptions d'outils précises, un format de réponse structuré et une séparation explicite entre les outils d'observation et les outils d'action.

5.6 Applications et cas d'usage

La solution réalisée a été validée à travers deux cas d'usage principaux. Ces scénarios ont été choisis car ils illustrent deux formes complémentaires d'exploitation des APIs CAMARA : d'une part, l'action sur le réseau à travers l'API Quality on Demand, et d'autre part, l'observation de signaux réseau pour l'aide à la décision dans un contexte de sécurité.

Ces deux cas d'usage permettent ainsi de valider le rôle du serveur MCP comme couche d'intermédiation entre les APIs CAMARA et les agents IA. Le MCP expose les capacités réseau sous forme d'outils, tandis que l'agent conserve la responsabilité du raisonnement, du choix des outils et de la décision finale.

5.6.1 Optimisation dynamique de la connectivité avec Quality on Demand

Le premier scénario repose sur l'API Quality on Demand. L'objectif est de permettre l'activation temporaire d'un profil de qualité de service adapté à une application critique, par exemple une visioconférence, un transfert de données sensible ou une session nécessitant une latence réduite.

Dans ce cas d'usage, l'utilisateur formule une demande en langage naturel, par exemple en demandant une meilleure qualité de connexion pour un terminal donné. L'agent analyse la demande et identifie qu'une action sur la qualité de service est nécessaire. Il appelle alors l'outil MCP correspondant à la création d'une session Quality on Demand.

Le serveur MCP transmet ensuite la requête au middleware CAMARA, qui se charge d'appeler l'endpoint associé. Une session QoD est créée avec les paramètres nécessaires, puis les informations de session sont retournées à l'agent. Celui-ci peut alors confirmer à l'utilisateur que la demande a été prise en compte et indiquer l'état de la session créée.

5.6.2 Évaluation du risque avec SIM Swap et Device Status

Le second scénario porte sur la détection de fraude dans le cadre d'une transaction sensible. L'objectif est d'exploiter des signaux télécoms pour aider à déterminer si une opération peut être considérée comme sûre ou si elle nécessite une vérification renforcée.

Dans ce cas d'usage, l'utilisateur demande si un numéro de téléphone est sécurisé pour une transaction. L'agent identifie qu'il doit réaliser une analyse de risque et appelle un outil MCP d'agrégation des signaux de sécurité. Cet outil interroge notamment l'API

SIM Swap afin de vérifier si un changement récent de carte SIM a eu lieu, ainsi que l'API Device Status afin d'obtenir l'état réseau du terminal.

Les résultats sont retournés à l'agent qui va ensuite analyser les signaux disponibles. Par exemple, un terminal joignable, connecté et sans changement récent de SIM peut conduire à une évaluation de risque faible. À l'inverse, un changement récent de carte SIM ou un état réseau incohérent peut augmenter le niveau de risque et conduire l'agent à recommander une vérification supplémentaire.

6 Compte rendu du projet

6.1 Déroulement général du projet

Le projet a été réalisé de manière progressive, en partant d'une étude théorique des APIs CAMARA jusqu'à la mise en place d'une chaîne technique complète reliant un environnement réseau 5G simulé, une couche CAMARA, un serveur MCP et un agent d'intelligence artificielle. L'objectif initial était de montrer comment des capacités réseau, habituellement manipulées par des composants techniques ou des applications déterministes, pouvaient être exposées sous forme d'outils et exploitées par un agent capable de raisonner sur un contexte.

La première phase du projet a consisté à étudier l'écosystème CAMARA et les APIs retenues pour les cas d'usage. Cette étape a permis d'identifier les fonctions nécessaires à la réalisation : Quality on Demand pour l'activation dynamique d'une qualité de service, SIM Swap pour la détection d'un changement récent de carte SIM, Device Status pour l'état du terminal, et Location pour la cohérence géographique. Cette phase a également permis de distinguer les APIs d'observation, qui fournissent des signaux au système, des APIs d'action, qui peuvent modifier l'état du réseau.

La deuxième phase a porté sur la mise en place de l'environnement réseau. Un cœur réseau free5GC a été déployé dans un environnement Docker, avec les principales fonctions du cœur 5G. UERANSIM a ensuite été configuré afin de simuler des terminaux mobiles et de vérifier leur enregistrement dans le réseau. Cette étape a été essentielle, car elle a permis de construire un environnement expérimental suffisamment réaliste pour tester les interactions entre les composants applicatifs et le réseau.

La troisième phase a consisté à intégrer une couche CAMARA basée sur les APIs officielles disponibles en ligne. Cette couche expose des endpoints REST correspondant aux capacités CAMARA étudiées, tout en assurant l'adaptation nécessaire avec l'environnement free5GC. Elle joue un rôle central dans le projet, car elle permet de masquer la complexité interne du cœur réseau et d'offrir une interface plus exploitable par les couches supérieures.

La quatrième phase a porté sur le développement du serveur MCP et de l'agent local. Le serveur MCP expose les capacités CAMARA sous forme d'outils structurés, tandis que l'agent local, exécuté via Ollama avec Llama 3.2, utilise ces outils pour analyser une demande, appeler les bonnes fonctions et produire une réponse contextualisée. Cette phase

a permis de valider l'idée principale du projet : le serveur MCP ne décide pas à la place de l'agent, mais lui fournit les moyens techniques d'interagir avec les APIs CAMARA.

Enfin, les dernières étapes ont concerné la validation des cas d'usage, l'amélioration du contrôle visuel du système et la préparation de la livraison finale. Une interface web de supervision a notamment été mise en place afin de visualiser les différents mobiles, leur état réseau, leur statut SIM Swap, leur profil QoS et le niveau de risque associé. Cette interface améliore le rendu de la démonstration, mais elle facilite aussi le contrôle technique du système, notamment pour vérifier l'état des sessions QoS sans devoir relancer manuellement des appels API à chaque étape.

6.2 Mise en place de l'environnement réseau et de la couche CAMARA

La première étape technique du projet a consisté à mettre en place un environnement réseau 5G expérimental. Pour cela, un cœur réseau free5GC a été déployé en Docker sur la première machine virtuelle. Cette architecture repose sur plusieurs conteneurs représentant les principales fonctions du cœur 5G, notamment AMF, SMF, PCF, UDM, UDR, NEF, NRF, UPF, AUSF, NSSF, MongoDB et l'interface WebUI. Ce déploiement permet de disposer d'un environnement complet pour simuler l'enregistrement de terminaux, la gestion des abonnés et l'accès à certaines informations internes du réseau. Une capture des conteneurs actifs est fournie en Annexe A.1.

Afin de simuler les terminaux mobiles, UERANSIM a été configuré et connecté au cœur free5GC. Cette étape a nécessité plusieurs ajustements, notamment autour de la configuration des slices réseau. Un problème lié à la valeur de slice 112233 empêchait initialement certains terminaux de s'enregistrer correctement. Après correction de cette configuration, les terminaux ont pu être enregistrés et visualisés dans la Web UI de free5GC. Cette validation est importante, car elle montre que le projet ne repose pas uniquement sur des réponses applicatives, mais bien sur un environnement réseau expérimental fonctionnel. La liste des UEs enregistrés est présentée en Annexe A.2.

Une fois l'environnement réseau opérationnel, une couche CAMARA a été déployée afin d'exposer les capacités réseau à travers des endpoints REST conformes aux APIs CAMARA étudiées. Cette couche repose sur les APIs officielles récupérées en ligne et adaptées à l'environnement expérimental du projet. Elle joue le rôle d'interface entre le monde applicatif et le cœur réseau free5GC, en évitant que les composants supérieurs aient à manipuler directement les APIs internes du cœur 5G. Le serveur CAMARA est exécuté dans un conteneur dédié, ce qui facilite son déploiement, son isolation et son intégration avec les autres composants. Une capture montrant le conteneur CAMARA actif est disponible en Annexe A.3.

La couche CAMARA intègre un mécanisme d'authentification OAuth2 reposant sur la génération d'un token JWT. Ce token est ensuite utilisé pour appeler les différentes routes exposées. Cette logique permet de conserver un comportement proche d'une API opérateur réelle, dans laquelle une application cliente doit d'abord s'authentifier avant d'accéder aux capacités réseau. Un exemple d'appel direct à l'API SIM Swap, avec récupération d'un

token puis vérification d'un numéro, est présenté en Annexe A.4.

Plusieurs capacités CAMARA ont été intégrées dans le projet. L'API SIM Swap permet de vérifier si un numéro a subi un changement récent de carte SIM. L'API Device Status permet d'obtenir l'état réseau d'un terminal, en s'appuyant notamment sur une conversion entre le numéro de téléphone et l'identifiant SUPI. L'API Location permet de manipuler des informations de localisation et d'évaluer une cohérence géographique. Enfin, l'API Quality on Demand permet de créer, consulter, prolonger ou supprimer des sessions QoS associées à un terminal.

Un point important du projet a été la liaison entre les informations applicatives CAMARA et les informations internes du cœur réseau. Par exemple, la conversion MSISDN vers SUPI permet de relier un numéro de téléphone à l'identifiant utilisé dans free5GC. De même, les profils 5QI récupérés ou représentés côté réseau ont été utilisés comme pont entre le monde 5G et les profils applicatifs QoS manipulés par CAMARA. Cette correspondance a permis de rendre les scénarios plus réalistes, notamment pour le cas d'usage Quality on Demand.

Certaines informations nécessaires aux cas d'usage n'étaient cependant pas directement disponibles dans free5GC. C'est notamment le cas de l'historique métier des changements de carte SIM ou de la localisation GPS réelle des terminaux. Pour répondre à cette limite, des données contrôlées ont été ajoutées dans l'environnement CAMARA, sous forme de registres associés aux abonnés. Cette adaptation ne remet pas en cause l'utilisation des APIs CAMARA : elle permet de tester la chaîne fonctionnelle complète dans un environnement expérimental, depuis la requête utilisateur jusqu'à l'appel CAMARA, puis l'exploitation de la réponse par le serveur MCP et l'agent d'intelligence artificielle.

6.3 Développement du serveur MCP

Après la mise en place de la couche CAMARA, le travail s'est concentré sur le développement du serveur MCP. Son rôle est de rendre les capacités CAMARA exploitables par un agent d'intelligence artificielle sous forme d'outils clairement définis. Le serveur MCP constitue donc l'interface entre l'agent et les APIs CAMARA : il reçoit les appels de l'agent, transmet les requêtes vers CAMARA, puis retourne les résultats dans un format structuré.

Plusieurs outils ont été exposés par le serveur MCP. Les outils `check_sim_swap`, `get_device_status`, `verify_location` et `retrieve_location` permettent de récupérer des informations liées au terminal, à la carte SIM ou à la localisation. Les outils `get_qod_profile`, `create_qod_session`, `get_qod_session`, `extend_qod_session` et `delete_qod_session` permettent quant à eux de manipuler les sessions Quality on Demand. La liste des outils disponibles est présentée en Annexe A.5.

Une distinction importante a été faite entre les outils d'observation et les outils d'action. Les outils d'observation récupèrent uniquement des signaux, par exemple l'état d'un terminal ou l'existence d'un SIM Swap récent. Les outils d'action peuvent modifier l'état du système, comme la création ou la suppression d'une session QoS. Cette séparation permet de mieux encadrer les décisions de l'agent et d'éviter qu'une action réseau soit

déclenchée sans intention explicite.

Le serveur MCP ne contient pas la logique métier de décision. Il ne décide pas si un paiement doit être accepté, ni si une session QoS doit être créée. Son rôle est d'exposer les capacités disponibles et de fournir des réponses exploitables par l'agent. La décision reste donc portée par l'agent IA, qui choisit les outils à appeler, interprète les résultats et formule la réponse finale.

Cette organisation permet également de rendre l'architecture indépendante du modèle utilisé. Le même serveur MCP peut être consommé par un agent local, exécuté avec Llama 3.2 via Ollama, ou par un agent utilisant un modèle externe plus puissant. Cette modularité constitue un point important du projet, car elle montre que la couche MCP peut rester stable même si le modèle d'intelligence artificielle évolue.

6.4 Intégration de l'agent local

L'étape suivante a consisté à intégrer un agent local capable d'utiliser les outils exposés par le serveur MCP. Cet agent repose sur Llama 3.2 exécuté avec Ollama, afin de conserver l'exécution du modèle dans l'environnement du projet. Ce choix est cohérent avec le contexte télécom, car les données manipulées peuvent contenir des numéros de téléphone, des états réseau ou des informations liées à une transaction.

Le fonctionnement de l'agent suit une boucle simple. L'utilisateur formule une demande en langage naturel, l'agent analyse l'intention, consulte les outils MCP disponibles, puis sélectionne celui qui correspond au besoin. L'appel est ensuite réellement exécuté par le client MCP, qui transmet la requête au serveur MCP. Le serveur appelle alors l'API CAMARA concernée et retourne une réponse structurée à l'agent. Une trace de cette exécution, avec un appel de l'outil `check_sim_swap`, est fournie en Annexe A.6.

Un point important de cette phase a été de vérifier que l'agent n'affichait pas seulement une intention d'appel, mais déclenchait réellement l'outil MCP. Les premiers tests ont montré qu'un modèle peut parfois produire un appel d'outil sous forme textuelle sans que l'exécution technique soit réalisée. La boucle d'exécution a donc été ajustée afin de bien distinguer la décision du modèle, l'appel effectif de l'outil et l'exploitation du résultat retourné.

Cette intégration confirme le rôle du serveur MCP comme intermédiaire contrôlé entre l'agent et CAMARA. L'agent conserve la responsabilité du raisonnement, tandis que le MCP assure l'accès technique aux APIs.

6.5 Intégration du second agent d'intelligence artificielle

Après avoir validé le fonctionnement du premier agent local, une limite importante a été constatée lors des tests. L'agent reposant sur Llama 3.2 via Ollama fonctionnait correctement sur le plan architectural, puisqu'il était capable d'interagir avec le serveur MCP et de déclencher des outils CAMARA. Cependant, ses performances restaient limitées par la puissance de calcul disponible sur la machine virtuelle. En pratique, certaines requêtes nécessitaient environ deux minutes avant d'obtenir une réponse exploitable. Ce délai est

problématique dans un contexte télécom, où les décisions liées à la qualité de service ou à la sécurité doivent être prises rapidement.

Au-delà du temps de réponse, la qualité des résultats produits par le premier agent n'était pas toujours suffisante. Certaines réponses manquaient de précision, et le modèle pouvait parfois produire des interprétations incomplètes ou inexactes des résultats retournés par les outils MCP. Cette limite est liée à la fois à la taille du modèle local, aux ressources matérielles disponibles et à la sensibilité générale des modèles de langage aux hallucinations. Dans un scénario de sécurité ou d'orchestration réseau, ce comportement doit être maîtrisé, car une mauvaise interprétation d'un signal CAMARA peut conduire à une décision peu pertinente.

Pour répondre à cette difficulté, un second agent a été implémenté sur le même serveur MCP, avec exactement les mêmes outils que ceux exposés au premier agent. L'objectif était de conserver une architecture strictement comparable : les capacités CAMARA, les routes appelées, les outils MCP et les réponses JSON retournées restent identiques. La seule différence porte sur le modèle utilisé pour analyser la demande, sélectionner les outils et interpréter les résultats.

Ce second agent repose sur le modèle **GPT-5.4 nano**, appelé via un token d'API. Ce choix a permis d'obtenir un agent plus réactif et plus fiable, sans modifier la couche MCP ni le middleware CAMARA. Grâce à la puissance du modèle distant, le prompt système a également pu être enrichi avec davantage de consignes : rappel du rôle de chaque outil, distinction entre observation et action, format de réponse attendu, justification des décisions et prudence dans les scénarios de sécurité. Contrairement au modèle local, cet enrichissement du prompt n'a pas entraîné de ralentissement significatif.

Les résultats obtenus avec le second agent sont nettement meilleurs. Les temps de réponse observés se situent généralement entre deux et cinq secondes par requête, ce qui est beaucoup plus compatible avec un contexte réseau ou télécom. Les réponses sont également plus pertinentes, mieux structurées et mieux justifiées. L'agent sélectionne plus correctement les outils MCP, explique davantage les raisons de ses choix et interprète de manière plus cohérente les signaux retournés par les APIs CAMARA.

Cette évolution confirme l'intérêt de l'architecture MCP. Le remplacement ou l'ajout d'un agent ne nécessite pas de modifier les APIs CAMARA ni les outils exposés. Le serveur MCP joue bien son rôle de couche stable entre les modèles d'intelligence artificielle et les capacités réseau. Il devient ainsi possible de comparer plusieurs agents, ou de choisir dynamiquement le modèle le plus adapté, sans remettre en cause l'architecture technique globale du projet.

6.6 Interface web de supervision et contrôle des mobiles

Une interface web de supervision a également été mise en place afin d'améliorer la lisibilité du système et de faciliter les démonstrations. Cette idée répond à une recommandation du tuteur : disposer d'un affichage centralisé des différents mobiles, de leurs informations réseau et de leur état fonctionnel.

L'interface permet notamment de visualiser les numéros de téléphone, les identifiants

associés, le statut du terminal, les informations SIM Swap, les profils QoS et le niveau de risque. Elle évite ainsi de devoir refaire manuellement des appels API pour vérifier l'état d'un terminal ou d'une session. Cette fonctionnalité est particulièrement utile pour Quality on Demand, car elle permet de contrôler plus rapidement si une session QoS a bien été créée ou mise à jour. Une capture de cette interface est présentée en Annexe A.9.

6.7 Validation des cas d'usage

6.7.1 Validation du cas Quality on Demand

Le premier cas d'usage validé concerne l'activation d'une session Quality on Demand. L'objectif était de vérifier que l'agent pouvait déclencher une action réseau à travers le serveur MCP et la couche CAMARA. Lorsqu'un besoin de connectivité renforcée est exprimé, l'agent sélectionne l'outil MCP adapté, puis transmet la demande de création de session QoS.

La requête est ensuite envoyée à l'API CAMARA, qui crée une session associée au terminal concerné. La réponse contient notamment un identifiant de session, un état et le profil QoS demandé. Une capture de création de session QoS est fournie en Annexe A.7. La session peut ensuite être relue à partir de son identifiant, ce qui permet de vérifier que l'action a bien été prise en compte. Un exemple de consultation d'une session existante est présenté en Annexe A.8.

Ce scénario valide le fonctionnement d'une API d'action. Contrairement aux APIs d'observation, Quality on Demand modifie l'état du système. Il était donc important de vérifier que l'action était déclenchée uniquement après une décision explicite de l'agent.

6.7.2 Validation du cas SIM Swap et sécurité d'un paiement

Le second cas d'usage concerne l'analyse d'un risque associé à une opération sensible, par exemple une transaction en ligne. Dans ce scénario, l'utilisateur demande si un numéro de téléphone peut être considéré comme sûr. L'agent identifie alors qu'il doit vérifier un signal de sécurité et appelle l'outil MCP `check_sim_swap`.

L'appel est transmis à la couche CAMARA, qui vérifie si un changement récent de carte SIM est associé au numéro concerné. Le résultat est ensuite retourné à l'agent sous forme structurée. Une trace montrant l'appel de l'outil MCP par l'agent est présentée en Annexe A.6. Un appel direct à l'API SIM Swap est également fourni en Annexe A.4.

Ce scénario valide l'utilisation d'une API d'observation. L'agent ne modifie pas l'état du réseau : il récupère un signal, l'interprète et produit une recommandation. Cette séparation entre observation et action constitue un point important de l'architecture réalisée.

6.8 Difficultés rencontrées et solutions apportées

Plusieurs difficultés techniques ont été rencontrées pendant la réalisation du projet. La première concernait la mise en place de l'environnement free5GC et l'enregistrement des

terminaux avec UERANSIM. Un problème de configuration lié à la slice réseau empêchait initialement certains terminaux de s'attacher correctement au cœur 5G. Cette difficulté a été résolue en corrigeant les paramètres de slice afin d'assurer la cohérence entre free5GC et UERANSIM.

Une autre difficulté importante concernait l'adaptation entre le monde applicatif CAMARA et les identifiants internes du cœur réseau. Les APIs manipulées côté application utilisent principalement des numéros de téléphone, tandis que free5GC s'appuie sur des identifiants comme le SUPI. Un mécanisme de conversion MSISDN vers SUPI a donc été intégré afin de relier correctement les requêtes CAMARA aux abonnés présents dans le réseau.

Enfin, certaines informations nécessaires aux scénarios, comme la localisation GPS réelle, ne sont pas fournies directement par free5GC. Des données contrôlées ont donc été ajoutées dans l'environnement de test afin de valider les cas d'usage sans s'éloigner de la logique CAMARA.

6.9 Fonctionnement technique de bout en bout

Le fonctionnement final du système repose sur une chaîne d'orchestration complète reliant l'utilisateur, l'agent d'intelligence artificielle, le serveur MCP, le middleware CAMARA et l'environnement réseau free5GC. L'objectif de cette architecture n'est pas seulement d'appeler des APIs réseau depuis un programme, mais de permettre à un agent IA de transformer une demande formulée en langage naturel en une série d'actions techniques contrôlées, traçables et interprétables.

Lorsqu'un utilisateur formule une demande, l'agent commence par analyser l'intention exprimée. Cette intention peut correspondre à une action sur la qualité de service, à une vérification de sécurité, à une récupération d'informations sur un abonné ou à une opération plus générale. À partir de cette analyse, l'agent sélectionne l'outil MCP le plus adapté. Cette étape est importante car l'agent ne manipule pas directement les endpoints REST CAMARA : il raisonne à partir d'outils fonctionnels décrits dans le serveur MCP.

Une fois l'outil sélectionné, l'appel est transmis au serveur MCP. Le serveur vérifie les paramètres fournis par l'agent, par exemple le numéro de téléphone, l'identifiant d'authentification OTP, le profil QoS demandé, la durée de session ou le type de vérification à effectuer. Le serveur MCP construit ensuite la requête HTTP correspondante vers le middleware CAMARA. Cette requête peut nécessiter un token d'accès obtenu via le mécanisme d'authentification OAuth2/JWT mis en place dans le middleware. Cette étape permet de conserver un fonctionnement proche d'une API opérateur réelle, dans laquelle les capacités réseau ne sont pas exposées sans contrôle d'accès.

Le middleware CAMARA reçoit ensuite la requête et effectue la traduction entre les paramètres applicatifs et l'environnement réseau expérimental. Dans plusieurs cas, il doit convertir un identifiant applicatif, comme un numéro de téléphone MSISDN, vers un identifiant réseau interne comme le SUPI utilisé dans free5GC. Cette correspondance est essentielle car les APIs CAMARA raisonnent principalement en termes applicatifs, tandis que le cœur réseau 5G manipule des identifiants techniques propres aux abonnés et aux

terminaux.

Pour les scénarios de qualité de service, le middleware CAMARA traite les opérations liées à Quality on Demand. Une demande de création de session QoD contient par exemple le terminal concerné, le profil de qualité demandé, la durée de validité et éventuellement le type d'usage applicatif. Le middleware crée alors une session associée à un identifiant, à un état et à un profil QoS. Dans le cadre du projet, ces profils sont reliés à la logique réseau à travers des profils 5QI, ce qui permet de rapprocher la demande applicative CAMARA des mécanismes de qualité de service utilisés dans les réseaux 5G.

Pour les scénarios de sécurité, le middleware CAMARA agrège plusieurs signaux réseau. L'API SIM Swap permet de vérifier si une carte SIM a été changée récemment. L'API Device Status permet de connaître l'état du terminal, par exemple sa joignabilité ou son état de connexion. L'API Device Location permet d'ajouter une vérification de cohérence géographique. L'API OTP SMS permet de déclencher une vérification active de possession du numéro par code à usage unique. Enfin, l'API Subscriber Profile permet de récupérer une synthèse structurée du profil de l'abonné à partir de son numéro de téléphone. Ces informations ne sont pas toutes directement disponibles dans free5GC ; certaines sont donc représentées dans des registres contrôlés du middleware CAMARA afin de permettre la validation complète du scénario dans un environnement expérimental.

Une fois la réponse CAMARA obtenue, le serveur MCP la reformate dans une structure JSON normalisée et exploitable par l'agent. Cette normalisation permet de réduire l'ambiguïté pour le modèle de langage : au lieu de recevoir une réponse brute difficile à interpréter, l'agent reçoit des champs structurés, comme l'état du terminal, le résultat du SIM Swap, l'état d'une session QoD, le statut d'un OTP ou les informations principales d'un profil abonné.

L'agent interprète ensuite ces résultats pour produire une réponse finale. Dans un scénario de paiement, il peut par exemple conclure que le risque est faible si aucun SIM Swap récent n'est détecté, si le terminal est joignable et si l'OTP a été validé correctement. À l'inverse, un changement récent de carte SIM, un terminal injoignable ou un OTP non validé peuvent conduire à recommander une vérification renforcée. Dans un scénario QoD, l'agent peut confirmer la création d'une session de qualité de service, restituer son identifiant, son état et le profil appliqué.

Cette architecture met donc en évidence une séparation claire des responsabilités. L'agent IA porte le raisonnement et la décision. Le serveur MCP expose les capacités disponibles sous forme d'outils contrôlés. Le middleware CAMARA traduit ces outils en appels REST et assure l'adaptation avec l'environnement réseau. Enfin, free5GC fournit le cœur réseau expérimental sur lequel reposent les scénarios de validation.

6.10 Ajouts réalisés en fin de projet : serveurs MCP spécialisés et nouvelles routes CAMARA

En fin de projet, l'architecture a été enrichie afin de mieux séparer les responsabilités et d'intégrer de nouvelles capacités CAMARA. Deux évolutions principales ont été réalisées :

la séparation du serveur MCP en serveurs spécialisés et l'ajout de nouvelles routes CAMARA liées à l'authentification OTP par SMS et au profil abonné.

6.10.1 Ajout de deux serveurs MCP spécialisés

Dans la première version du projet, l'ensemble des outils MCP était regroupé dans un serveur unique. Cette approche était suffisante pour valider le principe général de l'intégration entre un agent IA et les APIs CAMARA, mais elle présentait une limite importante : tous les outils étaient accessibles dans le même espace, y compris des outils de nature très différente. Un outil d'observation comme `check_sim_swap` était exposé au même niveau qu'un outil d'action comme `create_qod_session`.

Pour améliorer la modularité et mieux contrôler les actions disponibles, deux serveurs MCP spécialisés ont été ajoutés au serveur général existant. L'architecture finale repose donc sur trois serveurs MCP complémentaires : un serveur général, un serveur spécialisé dans la qualité de service et un serveur spécialisé dans la sécurité.

Le serveur MCP général, exposé sur le port 2020, conserve une vision globale du système. Il permet d'accéder à l'ensemble des capacités principales et sert de point d'entrée complet pour les démonstrations ou les scénarios nécessitant plusieurs familles d'APIs CAMARA.

Le serveur MCP spécialisé QoS, exposé sur le port 2224, regroupe uniquement les outils liés à Quality on Demand. Il expose notamment les fonctions permettant de récupérer les profils QoS disponibles, de créer une session QoD, de consulter une session existante, de la prolonger ou de la supprimer. Ce serveur est utilisé lorsque l'intention de l'utilisateur concerne l'amélioration temporaire de la connectivité, la priorisation d'un flux ou l'adaptation d'un profil réseau à une application critique.

Le serveur MCP spécialisé sécurité, exposé sur le port 2225, regroupe les outils liés aux scénarios de vérification et de prévention de la fraude. Il expose les fonctions associées à SIM Swap, Device Status, Device Location, OTP SMS et Subscriber Profile. Ce serveur est utilisé lorsque la demande de l'utilisateur porte sur l'évaluation d'un risque, la vérification d'un abonné ou la sécurisation d'une transaction.

Cette séparation présente plusieurs avantages techniques. Elle réduit d'abord le risque qu'un agent appelle un outil inadapté au contexte de la demande. Elle permet ensuite de limiter l'exposition des outils sensibles : un agent chargé uniquement d'une vérification de sécurité n'a pas besoin d'accéder aux fonctions de création ou de suppression d'une session QoD. Elle évite également la surcharge d'un serveur MCP unique en répartissant les appels selon la nature de la demande : les requêtes liées à la QoS sont orientées vers le serveur spécialisé QoS, les requêtes de sécurité vers le serveur spécialisé sécurité et les demandes transverses vers le serveur général. Enfin, elle facilite la maintenance du projet, car les outils QoS et les outils de sécurité peuvent évoluer indépendamment.

Le client agent peut ainsi sélectionner dynamiquement le serveur MCP à utiliser en fonction de l'intention détectée. Si la demande concerne la qualité de service, l'agent utilise le serveur QoS. Si la demande concerne une vérification de sécurité, il utilise le serveur sécurité. Si la demande est plus générale ou nécessite plusieurs familles de fonctionnalités,

il peut utiliser le serveur MCP global. Cette organisation rapproche l'architecture d'un modèle plus industriel, dans lequel les capacités techniques sont séparées par domaine fonctionnel.

6.10.2 Ajout des routes CAMARA OTP SMS

Une nouvelle famille de routes CAMARA a été ajoutée afin de gérer un mécanisme d'authentification par code OTP envoyé par SMS. Cette fonctionnalité complète les APIs déjà utilisées pour la sécurité, comme SIM Swap et Device Status. L'objectif n'est plus seulement d'observer l'état réseau d'un terminal, mais aussi de vérifier que l'utilisateur est bien en possession du numéro de téléphone associé à l'opération.

Trois endpoints ont été intégrés pour ce mécanisme :

Endpoint CAMARA	Méthode	Rôle technique
<code>/one-time-password-sms/v0/send-code</code>	POST	Crée une demande d'authentification OTP, génère un identifiant unique et simule l'envoi d'un code par SMS au numéro concerné.
<code>/one-time-password-sms/v0/validate-code</code>	POST	Vérifie que le code transmis par l'utilisateur correspond au code associé à la demande d'authentification OTP.
<code>/one-time-password-sms/v0/status/:authenticationId</code>	GET	Récupère l'état d'une authentification OTP à partir de son identifiant : en attente, validée, expirée ou échouée.

TABLE 3 : Routes CAMARA ajoutées pour l'authentification OTP par SMS

Le fonctionnement technique est le suivant. Lorsqu'un utilisateur ou un agent déclenche l'envoi d'un OTP, une requête POST est envoyée vers `/one-time-password-sms/v0/send-code`. Le middleware CAMARA génère alors un identifiant d'authentification, appelé `authenticationId`, et l'associe au numéro de téléphone concerné. Un code OTP est généré puis stocké temporairement dans l'environnement de test avec un état initial, par exemple `PENDING`. Dans un environnement opérateur réel, le code serait transmis par SMS ; dans notre environnement expérimental, l'envoi est simulé afin de tester la chaîne fonctionnelle.

Lorsque l'utilisateur fournit le code reçu, l'agent peut appeler l'outil MCP correspondant à la validation OTP. Cet outil transmet une requête POST vers `/one-time-password-sms/v0/validate-code`. Le middleware vérifie alors la correspondance entre l'`authenticationId`, le numéro de téléphone et le code fourni. Si le code est correct, l'état de la demande peut passer à `VALIDATED`. Si le code est incorrect, expiré ou associé à un identifiant inconnu, la validation échoue et le statut retourné permet à l'agent de recommander une vérification supplémentaire.

Le troisième endpoint, `/one-time-password-sms/v0/status/:authenticationId`, permet de consulter l'état courant d'une authentification OTP. Il est utile pour vérifier

si une demande est encore en attente, validée, expirée ou échouée. Cette route améliore la traçabilité du processus, car l'agent peut contrôler l'état d'une authentification sans relancer une nouvelle demande OTP.

L'ajout de cette API renforce le scénario de sécurité. SIM Swap permet de détecter un risque lié à un changement récent de carte SIM, Device Status permet d'observer l'état du terminal, mais OTP SMS ajoute une vérification active de possession du numéro. Le système peut ainsi combiner plusieurs niveaux de contrôle : observation du réseau, cohérence de l'état du terminal et validation explicite par code.

6.10.3 Ajout de la route CAMARA Subscriber Profile

Une seconde fonctionnalité CAMARA a été ajoutée avec la route de récupération du profil complet d'un abonné :

Endpoint	Méthode	Rôle
<code>/subscriber/v1/profile/:phoneNumber</code>	GET	Récupère les informations associées à un abonné à partir de son numéro de téléphone.

TABLE 4 : Route CAMARA ajoutée pour la récupération du profil abonné

Cette route permet d'obtenir une vision plus complète du contexte associé à un numéro de téléphone. Le middleware CAMARA reçoit le numéro fourni dans l'URL, puis recherche le profil correspondant dans l'environnement de test. Selon les informations disponibles, le profil peut inclure le numéro de téléphone, l'identifiant réseau associé, l'état de l'abonné, le statut du terminal, les informations SIM Swap, les profils QoS disponibles ou encore des métadonnées utiles aux scénarios de sécurité.

L'intérêt de cette route est de centraliser les informations abonné dans un endpoint unique. Sans cette route, l'agent doit appeler séparément plusieurs APIs pour reconstruire le contexte d'un utilisateur. Avec `/subscriber/v1/profile/:phoneNumber`, il peut récupérer une synthèse structurée du profil et décider ensuite s'il doit approfondir l'analyse avec des appels plus spécialisés, par exemple SIM Swap, Device Status, OTP ou Quality of Service.

Techniquement, cette route joue également un rôle de pont entre le monde applicatif et le cœur réseau. Le numéro de téléphone fourni par l'utilisateur est un identifiant simple et compréhensible au niveau applicatif. Le middleware peut ensuite le relier aux identifiants internes utilisés dans l'environnement 5G, comme le SUPI. Cette correspondance facilite les scénarios de démonstration, car l'agent peut raisonner à partir d'un numéro de téléphone sans exposer directement les détails internes de free5GC.

Dans le cadre du serveur MCP sécurité, cette route est particulièrement utile. Elle permet à l'agent de commencer par récupérer le profil global d'un abonné, puis de compléter l'analyse par des vérifications spécifiques. Par exemple, pour une transaction sensible, l'agent peut récupérer le profil abonné, vérifier l'état du terminal, contrôler l'absence de SIM Swap récent, déclencher une vérification OTP, puis produire un niveau de risque final.

Cette logique permet de passer d'une simple vérification isolée à une analyse multi-sigaux plus robuste.

6.10.4 Intérêt global des ajouts

Ces ajouts améliorent significativement la cohérence technique du projet. La séparation en serveurs MCP spécialisés rend l'architecture plus modulaire et plus contrôlable. Les nouvelles routes OTP SMS et Subscriber Profile enrichissent les scénarios de sécurité en ajoutant une vérification active de possession du numéro et une vision plus complète du profil abonné.

L'architecture finale ne se limite donc plus à deux cas d'usage simples autour de SIM Swap et Quality on Demand. Elle permet désormais de construire des scénarios plus proches d'un environnement opérateur réel : activation dynamique de qualité de service, consultation d'un profil abonné, vérification de l'état réseau, détection d'un changement de SIM, contrôle de localisation et validation OTP. Cette extension renforce l'intérêt du projet, car elle montre que l'approche CAMARA, MCP et agent IA peut être généralisée à plusieurs familles d'APIs télécom.

6.11 Livraison finale et bilan

À l'issue du projet, la solution livrée comprend un environnement free5GC fonctionnel, une couche CAMARA opérationnelle, trois serveurs MCP, deux modes d'agents d'intelligence artificielle et une interface web de supervision. L'architecture permet de relier un cœur réseau 5G expérimental à des APIs CAMARA exposées sous forme d'outils MCP, puis exploitées par un agent capable de raisonner sur le contexte de la demande.

Les trois serveurs MCP constituent une évolution importante de la version finale. Le serveur général permet de conserver un accès global aux capacités du système. Le serveur spécialisé QoS expose les outils liés à Quality on Demand. Le serveur spécialisé sécurité expose les outils liés à SIM Swap, Device Status, Device Location, OTP SMS et Subscriber Profile. Cette séparation améliore la modularité, la lisibilité et le contrôle des outils mis à disposition des agents.

Les cas d'usage initiaux ont été validés. Le premier concerne l'activation d'une session Quality on Demand pour améliorer temporairement la qualité de service d'un terminal ou d'une application critique. Le second concerne l'évaluation d'un risque de transaction à partir de signaux réseau, notamment SIM Swap et Device Status. Ces scénarios ont ensuite été enrichis avec deux nouvelles familles de fonctionnalités CAMARA : l'authentification OTP par SMS et la récupération du profil complet d'un abonné.

L'ajout de l'OTP SMS permet de compléter les signaux d'observation par une vérification active. Le système ne se limite plus à constater l'état réseau d'un terminal ou l'existence d'un changement de SIM : il peut aussi vérifier que l'utilisateur est bien en possession du numéro utilisé pour l'opération. L'ajout de Subscriber Profile permet quant à lui de centraliser les informations relatives à un abonné et de fournir à l'agent un contexte plus complet avant la prise de décision.

Le projet montre ainsi la faisabilité d'une architecture dans laquelle un agent d'intelligence artificielle peut exploiter des APIs télécoms standardisées via MCP. La solution reste expérimentale, notamment parce que certaines informations sont simulées dans le middleware CAMARA, mais elle démontre clairement l'intérêt d'une séparation entre les capacités réseau, la couche d'exposition CAMARA, les serveurs MCP spécialisés et la logique de décision portée par l'agent. Cette organisation rend l'architecture extensible et permet d'ajouter progressivement de nouvelles APIs CAMARA sans remettre en cause la structure générale du système.

6.12 Conclusion

Ce projet a permis de démontrer concrètement l'évolution des réseaux télécoms vers des infrastructures programmables, exposées sous forme d'APIs. À travers l'intégration des APIs CAMARA dans un environnement expérimental basé sur free5GC, nous avons montré qu'il est désormais possible de consommer certaines capacités réseau comme des services applicatifs. Le réseau n'est donc plus seulement une infrastructure de transport de données : il devient une plateforme programmable, capable d'exposer ses fonctions à des applications externes. Dans ce contexte, le concept de *Network-as-a-Service* ne reste plus uniquement théorique, mais devient une approche techniquement réalisable. Le projet a également montré l'intérêt d'intégrer de l'intelligence artificielle dans les mécanismes de décision réseau. L'objectif n'était pas de remplacer les règles techniques existantes, mais de démontrer qu'un agent IA peut apporter une couche de raisonnement supplémentaire lorsque la décision dépend du contexte. Certains scénarios réseau restent plus adaptés à des mécanismes déterministes simples, notamment lorsque l'action à réaliser est connue à l'avance et ne nécessite pas d'interprétation. En revanche, lorsqu'il faut combiner plusieurs signaux, évaluer un risque ou adapter une réponse à une situation particulière, l'usage d'un agent devient plus pertinent. Cette logique apparaît clairement dans le cas d'usage lié au SIM Swap et à la prévention de la fraude. Un changement récent de carte SIM ne suffit pas toujours, à lui seul, à conclure qu'une transaction est frauduleuse. L'agent peut au contraire prendre en compte plusieurs signaux complémentaires, comme le statut du terminal, sa joignabilité, la cohérence de sa localisation, la validation d'un OTP ou encore les informations du profil abonné. À partir de cette analyse multi-sigaux, il peut produire une évaluation du risque plus contextualisée qu'une simple règle isolée. Le même raisonnement s'applique au cas d'usage Quality on Demand. Plutôt que de définir à l'avance un comportement fixe pour chaque situation, l'agent peut interpréter le besoin exprimé par l'utilisateur, identifier le type d'application concerné, choisir le profil de qualité de service le plus adapté et déclencher l'action correspondante via CAMARA. Cette approche ouvre la voie à une gestion plus dynamique et plus flexible de la qualité de service, dans laquelle le réseau peut s'adapter aux besoins applicatifs sans imposer une configuration entièrement prédéfinie. L'architecture mise en place repose sur une séparation claire entre les différentes couches du système : le cœur réseau expérimental, le middleware CAMARA, les serveurs MCP spécialisés et les agents d'intelligence artificielle. Cette organisation rend le projet évolutif. De nouvelles APIs CAMARA peuvent être ajoutées sans modifier pro-

fondement l'architecture générale. De nouveaux agents peuvent également être testés sur les mêmes outils MCP afin de comparer leurs performances, leur fiabilité ou leur pertinence dans différents scénarios. Enfin, ce projet peut s'inscrire dans une démarche d'évolution progressive vers des environnements plus proches de réseaux opérateurs réels. L'étape suivante consisterait à connecter cette architecture à des cœurs réseau plus complets, à enrichir les données disponibles, à ajouter de nouvelles fonctionnalités CAMARA et à automatiser davantage la sélection des serveurs MCP selon l'intention de l'utilisateur. À plus long terme, une telle approche pourrait contribuer à la mise en place de mécanismes de décision réseau dynamiques, capables d'adapter les services télécoms en fonction du contexte, du niveau de risque, des besoins applicatifs et de l'état réel du réseau.

Références

- [1] Linux Foundation, “Camara project documentation,” 2024. [Online]. Available : <https://camaraproject.org>
- [2] GSMA, “Gsm open gateway initiative,” 2023. [Online]. Available : <https://www.gsma.com/open-gateway>
- [3] W. Jiang *et al.*, “Network function virtualization : A survey,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 1, pp. 350–378, 2016.
- [4] 3GPP, “System architecture for the 5g system,” 3rd Generation Partnership Project, Tech. Rep. TS 23.501, 2023.
- [5] X. Foukas, G. Patounas, A. Elmokashfi, and M. Marina, “Network slicing in 5g : Survey and challenges,” *IEEE Communications Magazine*, vol. 55, no. 5, pp. 94–100, 2017.
- [6] J. Ordonez-Lucena and F. Dsouza, “Pathways towards network-as-a-service : The camara project,” in *Proceedings of the ACM SIGCOMM Workshop on Network-Application Integration*. ACM, 2022.
- [7] 3GPP, “Common api framework for 3gpp northbound apis,” 3rd Generation Partnership Project, Tech. Rep. TS 23.222, 2022.
- [8] ETSI, “Multi-access edge computing (mec) framework and reference architecture,” *ETSI White Paper*, 2019.
- [9] M. Valdez, D. Carrera, and A. Antonopoulos, “Standardization initiatives and market approaches in edge federation,” in *IEEE Conference on Standards for Communications and Networking*. IEEE, 2022.
- [10] R. Direito, K. Trantzas, D. Gomes, C. Tranoris, and R. Aguiar, “Offering camara apis as a service,” in *IEEE INFOCOM Workshops*. IEEE, 2025.
- [11] X. Wang, L. Feng, R. Zhang, F. Zhou, H. Du, W. Li, D. Niyato, and A. Jamalipour, “Llm-empowered agentic ai for qoe-aware network slicing management in industrial iot,” *arXiv preprint arXiv :2512.20997*, 2025.
- [12] K. Arun and P. Goel, “Facilitating shared introspection in hierarchical multi-agent systems using the model context protocol,” in *2025 5th International Conference on Robotics, Automation, and Artificial Intelligence (RAAI)*. IEEE, 2025, pp. 747–750.
- [13] V. Tupe and S. Thube, “Demonstrating multi-agent collaboration via agent-to-agent and model context protocols : An it incident response case study,” in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. IEEE, 2025, pp. 74–78.

- [14] Model Context Protocol, “Model context protocol specification : Tools,” 2025, accessed : 2026-05-15. [Online]. Available : <https://modelcontextprotocol.io/specification/2025-03-26/server/tools>

A Annexes

A.1 Déploiement de free5GC

```
ubuntu@rsx218-prj1-01:~$ sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
| grep -Ei "camara|amf|smf|upf|nrf|udm|udr|ausf|pcf|nssf|nef|mongodb|webui|ueransim"
camara      Up 3 days      0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
ueransim    Up 8 weeks     8000/tcp
udr          Up 2 months    8000/tcp
nssf        Up 2 months    8000/tcp
nef         Up 2 months    8000/tcp
webui       Up 2 months    0.0.0.0:2121-2122->2121-2122/tcp, [::]:2121-2122->2121-2122/tcp, 0.
0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
ausf        Up 2 months    8000/tcp
amf         Up 8 weeks     8000/tcp
pcf         Up 2 months    8000/tcp
udm         Up 2 months    8000/tcp
smf         Up 2 months    8000/tcp
nrf         Up 2 months    8000/tcp
mongodb     Up 2 months    27017/tcp
upf         Up 2 months
```

FIGURE 2 : Conteneurs free5GC actifs dans l'environnement Docker

A.2 Terminaux enregistrés dans free5GC

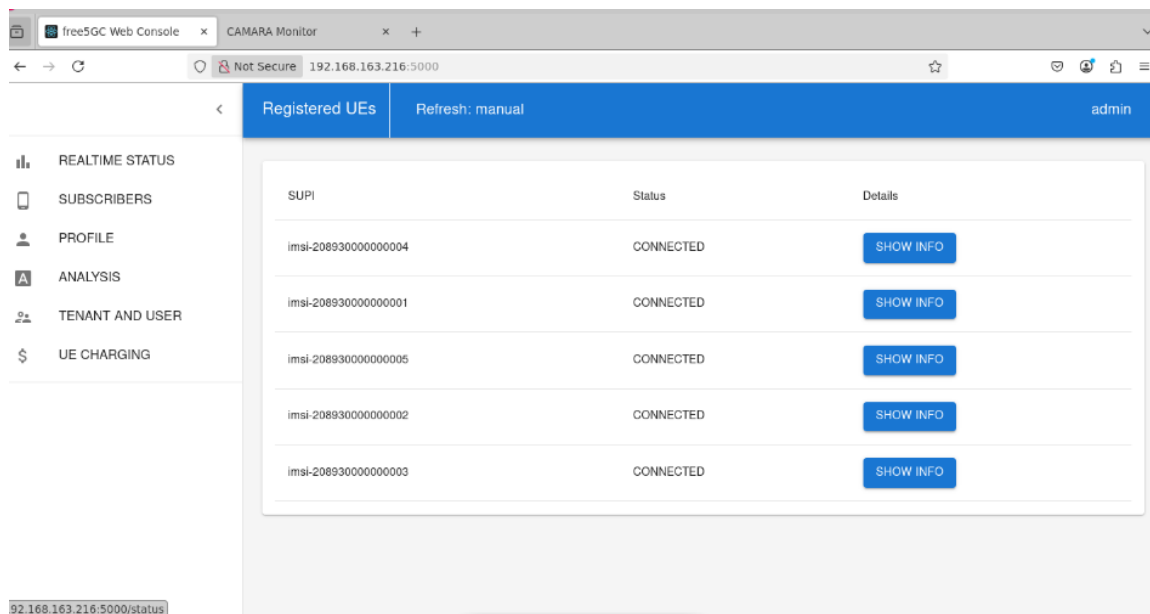


FIGURE 3 : Terminaux mobiles simulés enregistrés dans la Web UI de free5GC

A.3 Conteneur du serveur CAMARA

```
ubuntu@rsx218-prj1-01:~$ sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"  
| grep camara  
camara      Up 3 days      0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

FIGURE 4 : Conteneur Docker du serveur CAMARA exposé sur le port 3000

A.4 Appel direct à l'API SIM Swap CAMARA

```
ubuntu@rsx218-prj1-01:~$ TOKEN=$(curl -s -X POST http://localhost:3000/oauth/token \  
> -H "Content-Type: application/x-www-form-urlencoded" \  
> -d "client_id=test-client" \  
> -d "client_secret=test-secret" \  
> -d "scope=" | jq -r .access_token)  
ubuntu@rsx218-prj1-01:~$ curl -s -X POST http://localhost:3000/sim-swap/v1/check -H "Auth  
rization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{"phoneNumber":"0900000  
001","maxAge":240}' | jq  
{  
  "swapped": false,  
  "swapDate": null,  
  "source": "free5gc-udr",  
  "supi": "imsi-208930000000001",  
  "checkedAt": "2026-05-18T18:12:48.078Z"  
}  
ubuntu@rsx218-prj1-01:~$ █
```

FIGURE 5 : Récupération d'un token OAuth2 puis appel direct à l'API SIM Swap

A.5 Outils exposés par le serveur MCP

```
Vous: tools

Tools MCP disponibles :
- check_sim_swap
- get_device_status
- verify_location
- retrieve_location
- get_qod_profile
- create_qod_session
- get_qod_session
- extend_qod_session
- delete_qod_session
Vous: █
```

FIGURE 6 : Liste des outils exposés par le serveur MCP

A.6 Appel d'un outil MCP par l'agent local

```
Vous: mon téléphone 0900000001 est-il sécurisé pour un payment?

[Tool call] check_sim_swap({'phone_number': '0900000001', 'max_age': 24})
[Tool result] {'swapped': False, 'swapDate': None, 'source': 'free5gc-udr', 'supi': 'imsi-208930000000001', 'checkedAt': '2026-05-18T18:18:43.852Z'}

Agent: Votre téléphone 0900000001 n'a pas récemment subi un SIM Swap, ce qui est bon pour les paiements. Cependant, il est toujours recommandé de vérifier l'état réseau et la joignabilité du téléphone pour garantir sa sécurité.
Vous: █
```

FIGURE 7 : Exécution de l'outil `check_sim_swap` par l'agent local via le serveur MCP

A.7 Création d'une session Quality on Demand

```
ubuntu@rsx218-prj1-01:~$ SESSION_ID=$(curl -s -X POST http://192.168.163.216:3000/quality-on-demand/v1/sessions \
> -H "Authorization: Bearer $TOKEN" \
> -H "Content-Type: application/json" \
> -d '{
>   "device": {
>     "phoneNumber": "0900000002"
>   },
>   "qosProfile": "QOS_E",
>   "duration": 3600
> }' | jq -r '.sessionId // .id')
ubuntu@rsx218-prj1-01:~$
ubuntu@rsx218-prj1-01:~$ echo $SESSION_ID
8b64feff-4288-4a66-93d9-f392f73883dc
```

FIGURE 8 : Création d'une session Quality on Demand via l'API CAMARA

A.8 Consultation d'une session Quality on Demand

```
ubuntu@rsx218-prj1-01:~$ curl -s -X GET http://192.168.163.216:3000/quality-on-demand/v1/sessions/$SESSION_ID \
> -H "Authorization: Bearer $TOKEN" | jq
{
  "sessionId": "8b64feff-4288-4a66-93d9-f392f73883dc",
  "phoneNumber": "0900000002",
  "qosProfile": "QOS_E",
  "status": "AVAILABLE",
  "startedAt": "2026-05-18T18:23:29.864Z",
  "expiresAt": "2026-05-18T19:23:29.864Z"
}
ubuntu@rsx218-prj1-01:~$ █
```

FIGURE 9 : Consultation d'une session Quality on Demand à partir de son identifiant

A.9 Interface web de supervision CAMARA

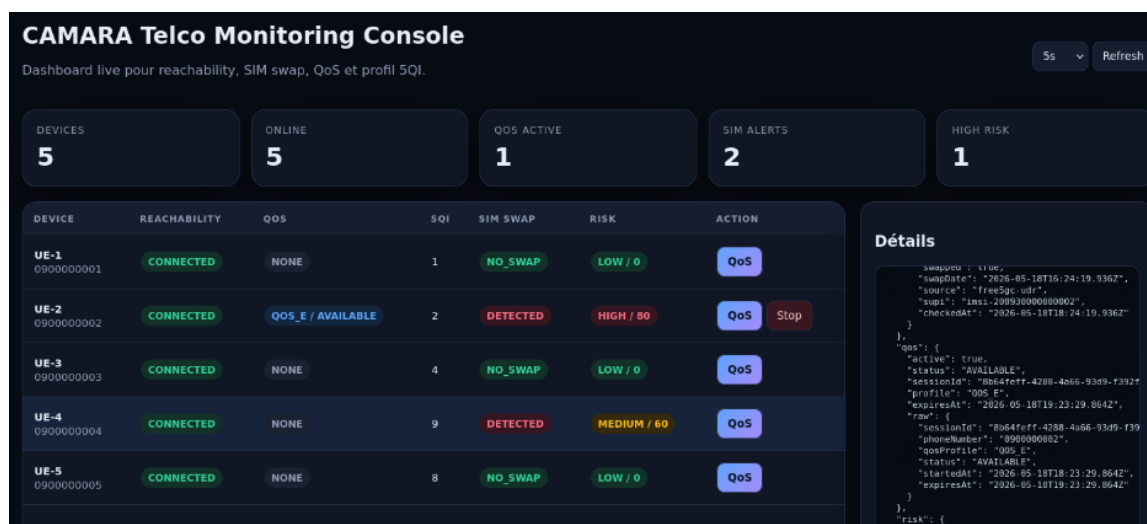


FIGURE 10 : Interface web de supervision des mobiles, des statuts SIM Swap, des profils QoS et du niveau de risque